강의 정리본

▼ DB01

Data 데이터
Known facts that can be recorded and that have implicit meaning **저장**될 수 있고 내제된 **의미**를 가진 알려진 **사실**

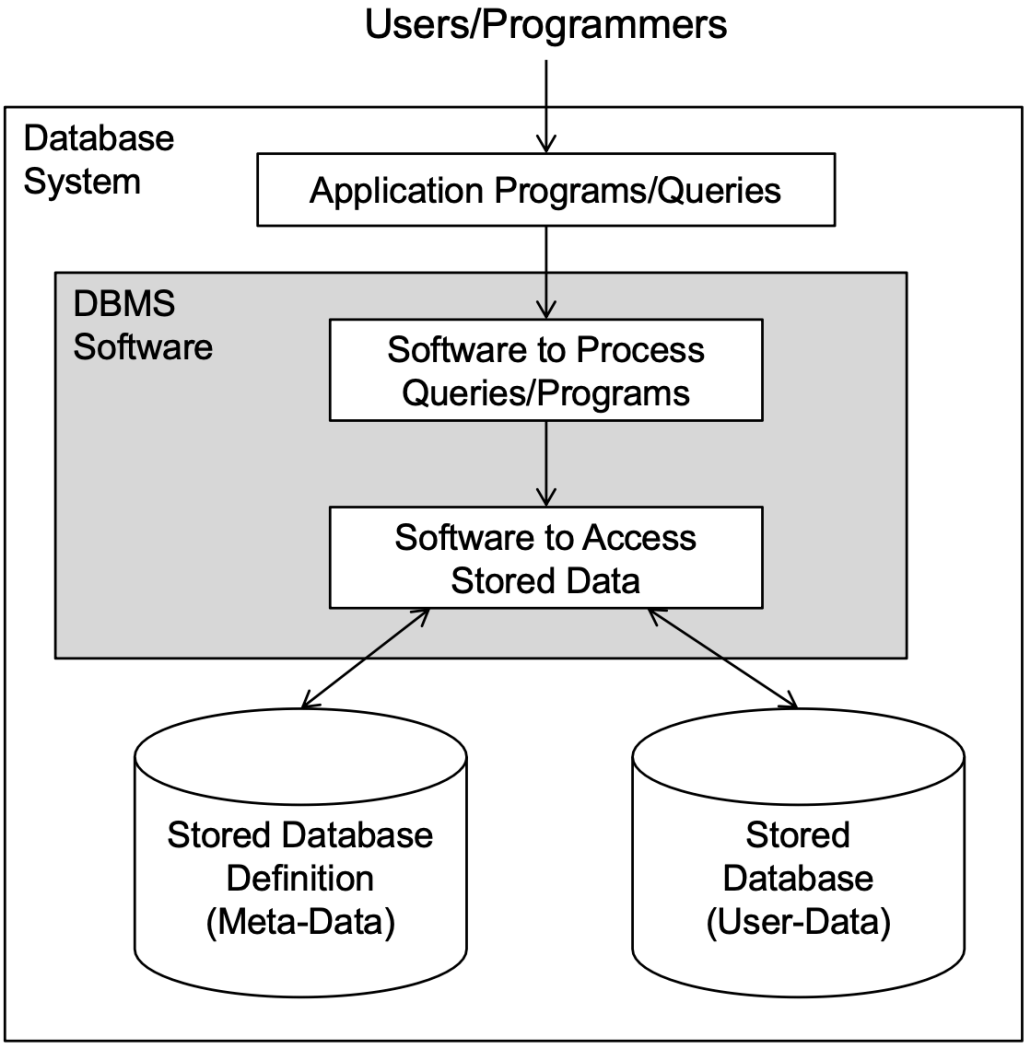
Database 데이터베이스
Logically coherent collection of **related data** 논리적으로 **관련된 데이터**의 집합

Mini-world
Represents some aspect of real world 실제 세계의 일부를 표현한 세계

Database Management System (DBMS)
Enables a user to **create** and **maintain** a database 사용자가 데이터베이스를 **생성 및 관리**하는데 사용하는 **소프트웨어**

Database System = Database + DBMS

데이터베이스 시스템의 구조



데이터베이스 예시 - 대학교 데이터베이스

- 대학을 운용하기 위해 필요한 데이터를 저장한다: 학생, 수업, 교수, 학점 등
- 이 데이터 간의 관계를 저장한다.

Data records

1. 학생: 이름, 학번, 전공, 반 등
2. 강의: 학수번호, 학점, 학부 등
3. 섹션: 섹션 ID, 학수번호, 학기, 연도, 교수 등
4. 성적표: 학번, 섹션 ID, 평점 등
5. 선수강 요건: 학수번호와 선수강 강좌 간의 관계

데이터베이스 특징

Self-describing nature

자기 자신의 구조를 기술하는 특성이 있다.

> **메타데이터 Meta-data**
>
> - 데이터베이스의 구조를 설명
> - 사용자 데이터를 관리하기 위해 필요한 데이터

System Catalog 시스템 카탈로그: 메타데이터를 저장하는 공간

데이터베이스 시스템은 사용자 데이터와 메타데이터 모두 저장/관리한다.

프로그램과 데이터 간의 단절

- Self-describing nature 덕분에 가능
- (사용자) 데이터 구조의 변화가 프로그램에 영향을 주지 않는다.

  사용자 데이터 구조의 변화는 메타데이터에 반영된다.
- 프로그램 - 데이터 간의 독립성을 제공

Data Abstraction 데이터 추상화

Hides the details of a data structure 실제 데이터 구조의 상세정보를 숨긴다.

Data model 데이터 모델: provides only a conceptual view rather than a physical view 물리적인 데이터 구조보다는 추상적인 뷰를 제공

View

실제 데이터베이스의 Virtual data 가상적인 데이터.

사용자에게 같은 데이터베이스에 대해 다른 관점을 제공

데이터 공유

데이터베이스는 하나이고, 여러 사용자가 동시에 접근할 수 있다.

일관성 문제를 방지하는 매커니즘 concurrency control mechanism 덕분에 가능

Users of Database Systems 데이터베이스 시스템의 사용자

1. System analyst 시스템 분석가: End users 최종 사용자의 요구사항(사용자가 필요한 데이터)을 분석
2. Database designer 데이터베이스 설계자: 시스템 분석가가 분석한 데이터 후보 중 저장해야 할 데이터를 선택하고, 데이터베이스의 구조를 설계
3. Application programmer 앱 개발자: 위에서 결정된 기능을 프로그램(사용자 인터페이스 등)으로 구현
4. Database administrator(DBA) 데이터베이스 관리자: 데이터베이스 시스템을 관리/모니터링
5. End users 최종 사용자: 데이터베이스 애플리케이션 프로그램을 통해 데이터베이스에 접근하는 사람

Workers behind the Scene

1. DBMS system designer & implementer: DBMS 컴포넌트 모듈과 인터페이스를 소프트웨어 패키지로 설계 및 구현
2. Tool developer: 데이터베이스를 편리하게 접근할 수 있는 소프트웨어 기능을 설계 및 구현
3. Operators and maintenance personnel: 데이터베이스 시스템의 운영을 담당

DBMS Features

1. Control redundancy 중복성 관리

> **Redundancy 중복성**
> 동일한 데이터가 여러 곳에 저장
>
> - 문제 1: 데이터 저장 공간 낭비
> - 문제 2: Data consistency 데이터 일관성 보장 불가

2. Restrict unauthorized access 권한이 없는 접근 방지
3. Persistent storage for program objects 프로그램의 종료와 관계없이 지속적으로 데이터를 저장
4. Multiple user interfaces 여러 사용자 수준을 위한 인터페이스 제공

   e.g. query language (SQL), 프로그램 인터페이스, GUI 등
5. Represent complex relationships among data 데이터 간의 복잡한 관계 표현
6. Enforce integrity constraints

   > Integrity 무결성
   > 데이터가 조건을 만족(결함이 없음)하는 특성

   → DBMS는 이러한 무결성의 조건을 정의할 수 있고, 해당 조건을 만족하도록 보장할 수 있다.
7. Backup & Recovery 백업과 복구
   a. 하드웨어/소프트웨어 고장으로부터 복구할 수 있게 해줌
   b. Backup: 데이터베이스의 운영 도중에 DBMS 내부에 데이터베이스의 복사본을 저장
   c. Recovery: 고장이 일어났을 때 최신 백업 데이터를 가져와 데이터베이스를 복구

DBMS의 장점

- 애플리케이션 개발 시간 절약: 데이터 관리를 위한 프로그래밍이 필요 없다.
- 최신 정보 제공: 어떤 한 사용자의 데이터베이스 업데이트가 모든 사용자에게 즉각적으로 보여진다.
- 규모의 경제: 사용자가 많아질수록 비용 절감

DBMS를 사용하면 안되는 경우?

Overheads of DBMS

- 높은 초기 비용: 하드웨어, 소프트웨어, 교육 등
- 높은 계산(시간) 비용: 보안, 일관성 제어, 무결성 조건 등

굳이 DBMS를 사용하지 않아도 되는 경우:

- 단순하고 앞으로 바뀔 가능성이 없는 애플리케이션
- Very strict real-time requirements: 마감기한이 중요한 애플리케이션

  e.g. 마감기한 내에 끝내야만 의미가 있고, 그렇지 않으면 의미가 없는 경우. 예를 들어 미사일 공격에 대한 대응을 n초 내에 해야 의미가 있음.
- 여러 사용자가 데이터베이스에 접근(쓰기)하는 경우가 없는 애플리케이션

▼ DB02

> Data Model 데이터 모델
> 데이터베이스의 구조를 표현하기 위한 개념의 집합
> 데이터 모델은 data abstraction 데이터 추상화를 제공한다.

개념의 레벨에 따라 3가지로 구분된다.

Physical Data Models 물리적 데이터 모델

실제 저장장치에 데이터가 물리적으로 어떻게 저장되는가? → Low-level 데이터 모델

Conceptual Data Models 개념적 데이터 모델

사용자에게 보여주기 위해 데이터가 어떻게 표현되는지 개념적으로 보여줌 → High-level 데이터 모델

e.g. Entity-relationship model

Representational Data Models 표현적 데이터 모델

물리적 데이터 모델과 개념적 데이터 모델 사이의 데이터 모델

- 사용자가 이해하기 쉬우면서 저장장치에 데이터가 어떻게 저장되어 있는지도 보여준다.
- Implementation Data Models라고도 부른다.
- 대부분의 상용 DBMS에서 사용된다.

e.g. Relational model, Network model, Hierarchical model

Schema

Database Schema 데이터베이스 스키마 (또는 메타데이터)
모델에 의해 표현된 데이터베이스의 구조.

- Intension이라고도 부른다.
- 데이터베이스를 설계할 때 결정되고, 데이터베이스 주기 동안 거의 변하지 않는다.

Schema Diagram

스키마를 시각적으로 표현한 자료.

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

PREREQUISITE

Course_number	Prerequisite_number

SECTION

Section_identifier	Course_number	Semester	Year	Instructor

GRADE_REPORT

Student_number	Section_identifier	Grade

Instance

Database Instance 데이터베이스 인스턴스

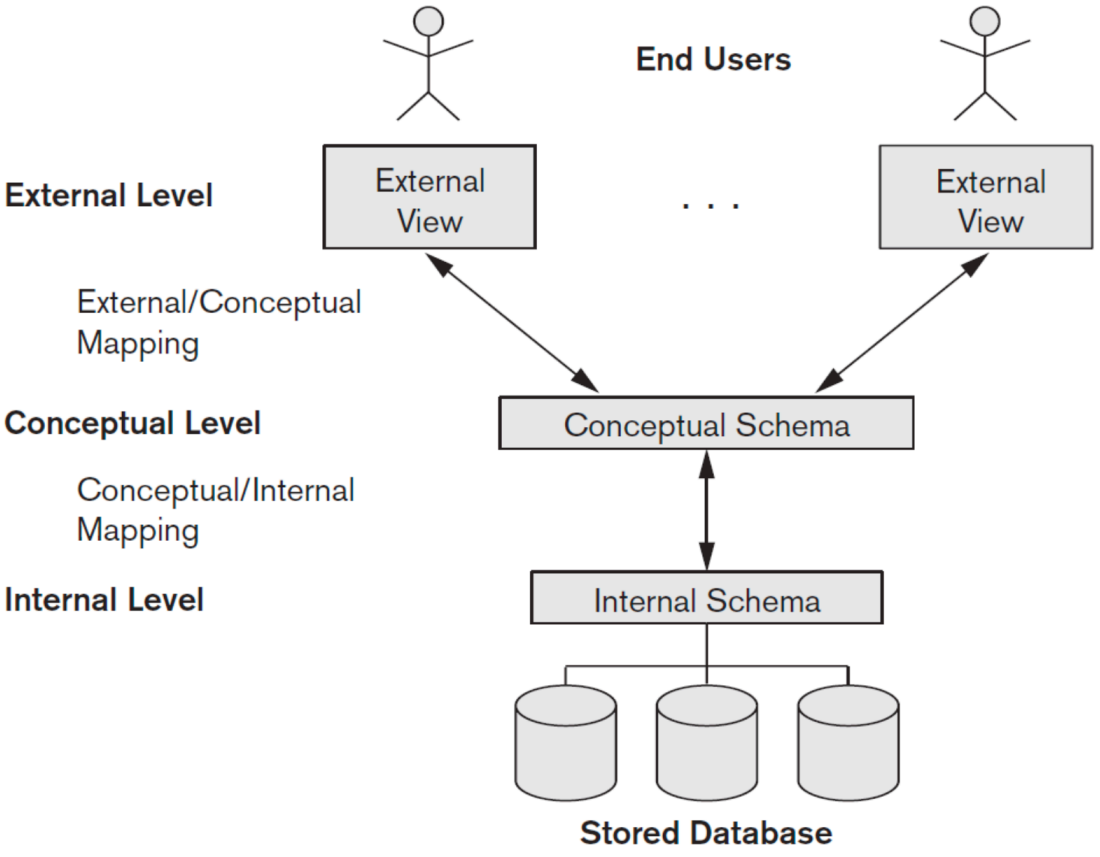
특정 순간에 데이터베이스에 저장되어 있는 데이터.

- Database state, snapshot, occurrence, extension이라고도 부른다.
- 데이터베이스 인스턴스는 삽입, 수정, 삭제 등에 의해 자주 바뀐다.

Three-Schema Architecture

> 목적: 애플리케이션 프로그램이 데이터베이스 구조에 종속되지 않게 하기 위함 → 독립성

데이터베이스 설계자가 데이터베이스에 저장할 데이터를 선택하고, 데이터베이스의 구조와 특징(schema)을 결정한다.



Internal Schema 내부 스키마

Low-level; 물리적 데이터 모델로 표현된다.

Conceptual Schema 개념 스키마

High-level; 개념적/표현적 데이터 모델로 표현된다.

External View 외부 뷰

각 사용자가 바라보는 데이터베이스의 일부분이다.

내부 스키마와 개념 스키마는 모두 **전체 데이터베이스에 대한 스키마**고, 개념 스키마와 외부 뷰는 **동일한 모델**로 표현된다.

- External/conceptual mapping: 외부 뷰에는 데이터베이스의 일부만 보여지기 때문에 사용자가 쿼리 요청을 하면 개념 스키마에 있는 전체 데이터베이스를 외부 뷰에 매핑하는 것.
- Conceptual/internal mapping: 개념 스키마에 내부 스키마를 매핑하는 것.

> 📌 **독립성**
> 만약 내부 스키마에 변화가 발생하더라도, 내부 스키마에 해당하는 메타데이터만 바뀌면 되기 때문에 사용자 프로그램에 영향을 주지 않는다.

Internal Level

전체 데이터베이스의 물리적 저장 구조를 표현한다.

→ Low-level 데이터 모델인 Physical data model로 표현된다.

Conceptual Level

전체 데이터베이스의 논리적 구조를 표현한다: 엔티티, 데이터 유형, 관계, 사용자 기능, 제약 조건 등

물리적 저장 구조를 감춰준다.

→ Higher level 데이터 모델로 표현된다: Conceptual & Representational

External/View Level

특정 사용자를 위한 데이터베이스의 일부분을 표현한다. 전체 데이터베이스의 나머지 부분은 감춘다.

→ Higher level 데이터 모델로 표현된다: Conceptual & Representational

External과 internal schema는 데이터베이스를 다른 방식으로 표현한다.

- Internal schema 내에는 물리적으로 데이터가 어떻게 저장되어 있는지 표현되어 있다.
- External schema 내에는 논리적 데이터베이스, 즉 논리적 모델에 의해 표현된 데이터베이스를 표현한다.

Mapping

1. 사용자 쿼리는 External schema에 대한 것
2. External schema 쿼리 → Conceptual schema로 매핑
3. Conceptual schema 쿼리 → Internal schema로 매핑
4. 데이터베이스에서 결과를 가져옴
5. 데이터베이스에서 가져온 결과를 External view에 매핑

Data Independence 데이터 독립성

상위 스키마를 바꿀 필요 없이 데이터베이스의 특정 단계의 스키마를 바꿀 수 있다는 특성이다.

Logical Data Independence

Conceptual schema를 External schema의 변경 없이 바꿀 수 있다.

→ External schema를 참조하는 애플리케이션 프로그램을 재작성할 필요가 없다.

Physical Data Independence

Internal schema를 Conceptual schema와 External schema의 변경 없이 바꿀 수 있다. 따라서 사용자 애플리케이션 프로그램을 다시 만들 필요 없이 Internal schema를 바꿀 수 있다.

→ DBMS를 사용하는 가장 큰 장점이다. 기존 애플리케이션 프로그램의 유지 비용을 크게 절감할 수 있다.

DBMS Languages

DBMS가 제공하는 언어. 데이터베이스를 제어/관리하는 언어다.

1. Data definition language (DDL): 데이터베이스의 골격과 관련된 언어
2. Data manipulation language (DML): 사용자의 데이터 삽입, 삭제, 수정 등의 기능 제공
3. Storage definition language (SDL): 데이터베이스의 Internal schema를 관리하는 언어
4. View definition language (VDL): 사용자의 뷰를 관리하는 언어

DDL, DML, SDL, VDL은 모두 하나의 SQL 안에 포함되어 있다.

DML은 크게 2가지 유형으로 나눌 수 있다:

1. Procedural DML

   사용자 쿼리를 **절차적**으로 정의한다. 사용자 쿼리를 처리할 순서를 하나하나 step-by-step 정의한다는 의미다. 데이터를 가져오기 위해서 어떻게 해야 하는지 how to retrieve data에 초점이 맞춰져 있다.

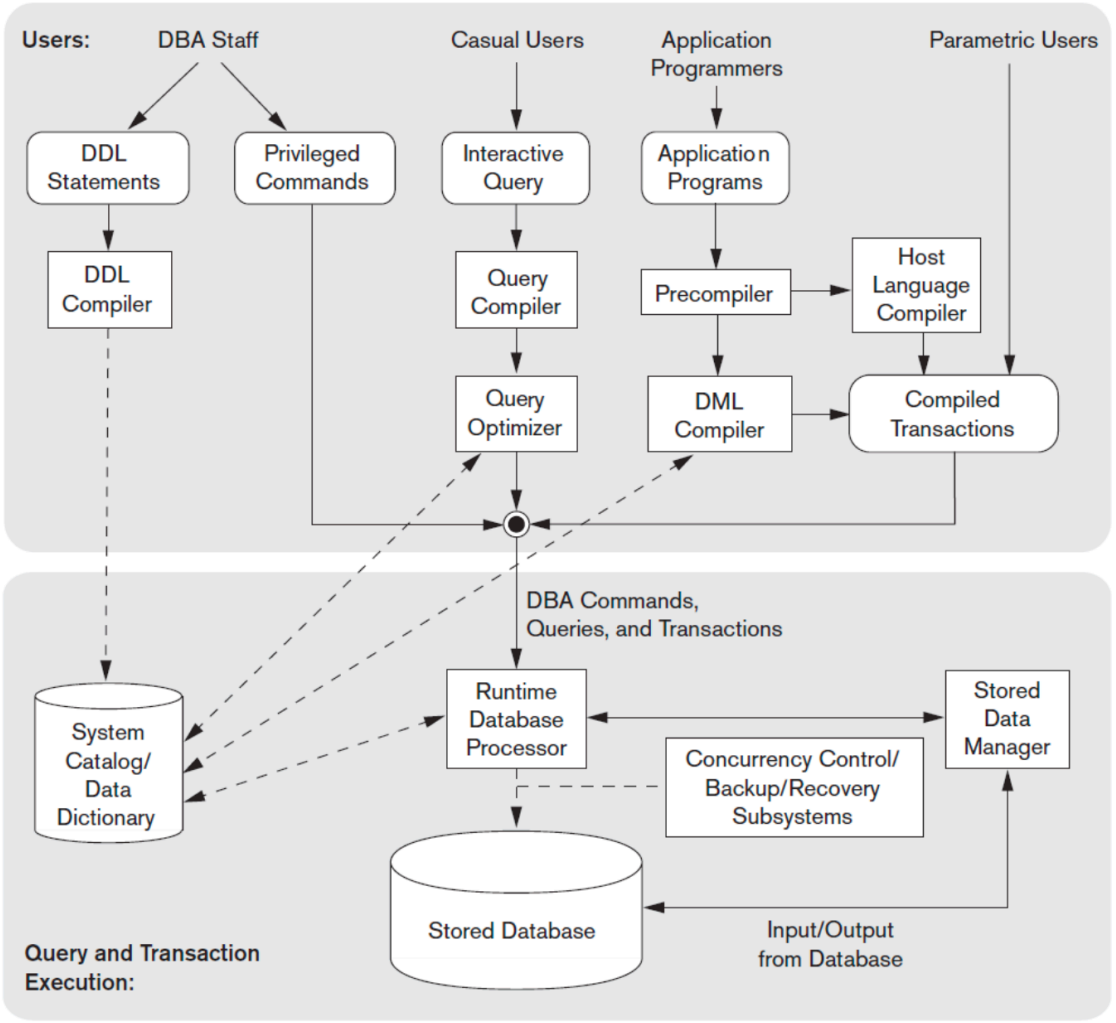
2. Non-procedural DML

   사용자 쿼리를 **선언적**으로 정의한다. 어떤 데이터를 가져올 것인지 what data to retrieve에 초점이 맞춰져 있다.

DBMS Interfaces 인터페이스

- Menu-based interfaces: 메뉴에서 제공하는 리스트를 통해 쿼리를 선택하여 데이터베이스에 접근하는 방식의 인터페이스. 주로 웹 애플리케이션에서 사용된다.
- Graphical user interfaces (GUI): 시각적으로 표현된 데이터베이스를 조작하여 데이터베이스에 접근하는 방식의 인터페이스. 주로 naive한 사용자를 위해 제공된다.
- Forms-based interfaces: 미리 정의된 양식을 채워 데이터베이스에 접근하는 방식의 인터페이스.
- Natural language interfaces: 자연어 인터페이스를 통해 데이터베이스에 접근하는 방식의 인터페이스.
- Interfaces for DBAs: 데이터베이스 관리자를 위한 인터페이스. DBMS 관리 환경을 위한 인터페이스다.

DBMS Component Modules



- Stored data manager: 저장장치에 저장된 모든 데이터(user data, meta data) 접근을 제어한다.
- DDL compiler: DDL에 의해 생성된 스키마를 처리하는 역할.
- Runtime database processor: 사용자 쿼리를 처리하기 위해 데이터베이스에 접근한다.
- Query compiler (interactive SQL): SQL로 표현된 쿼리를 파싱하여 데이터베이스가 이해할 수 있는 함수 호출로 바꿔주는 역할.

  SQL(or DBMS queries) (High-level) → DBMS language (Low-level)
- Precompiler: SQL로 짜여져 있는 부분을 골라내는 역할.
- DML compiler: DML을 컴파일하여 low-level DBMS language로 변환하여 데이터베이스에 접근할 수 있게 해준다.

Database System Utilities 데이터베이스 시스템 유틸리티

DBA가 쉽게 데이터베이스 시스템을 관리할 수 있게 해주는 편의 기능

- Loading: 데이터베이스 외부의 파일을 데이터베이스 포맷에 맞게 변환해주는 기능이다. 초기 데이터베이스 구축 시에 유용하다.
- Backup: 데이터베이스의 복사본을 백업하는 기능이다.
- Database storage reorganization: 데이터베이스의 구조를 재구성하는 기능이다. 기존 DBMS의 성능을 향상시키기 위해 사용한다.
- Performance Monitoring: 데이터베이스 사용량을 제공하여 DBA가 파일 재구성 여부를 판단할 수 있게 해준다.

DBMS Classification 분류

DBMS는 다양한 기준에 따라 분류할 수 있다.

Model 모델

- Relational model: 데이터베이스가 테이블 **table**의 집합으로 표현되는 모델. 각 테이블에는 record가 저장된다.
- Network model: 데이터를 record type과 1:N set type으로 표현하는 모델.
- Hierarchical model: 데이터를 계층적 트리 구조로 표현하는 모델.
- Object model: 데이터베이스가 class의 집합으로 표현되는 모델. 각 데이터는 객체 object로 표현되고, class에 소속된다.

Number of Users 사용자 수

- Single user DBMS
- Multi-user DBMS

Number of Sites

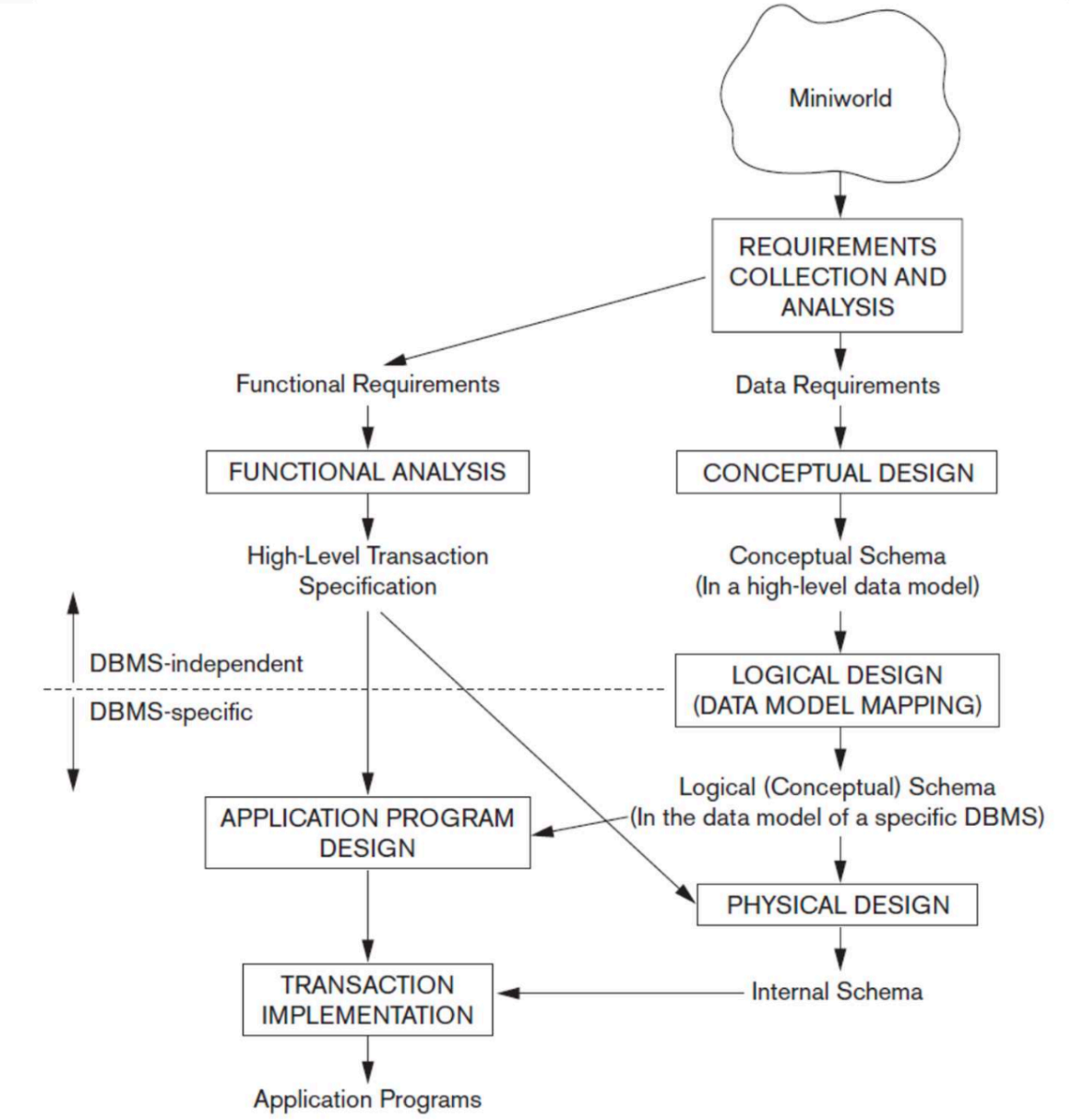
- Centralized DBMS 중앙집중형
- Distributed DBMS 분산형

- Homogeneous DBMS: 분산된 DBMS의 모델이 하나로 통일된 경우
- Heterogeneous DBMS: 분산된 DBMS의 모델이 서로 다른 경우

Purpose 목적

- General purpose
- Special purpose

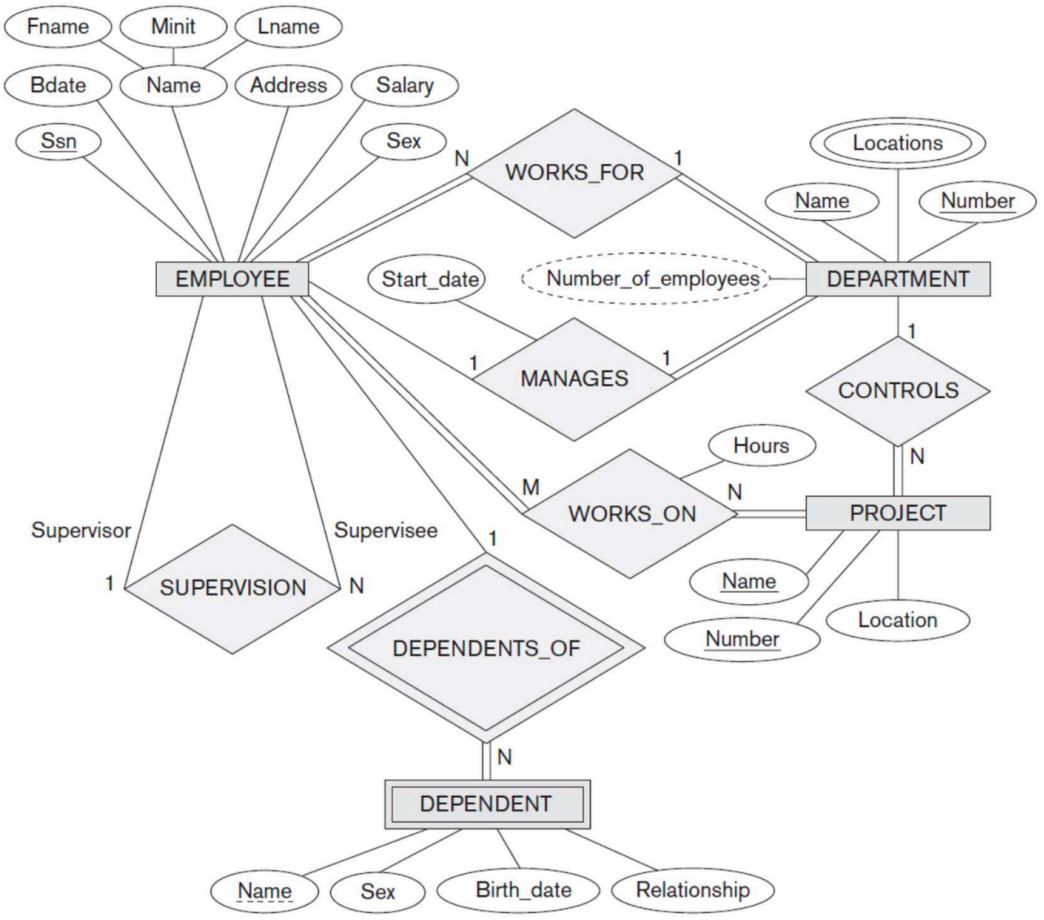
▼ DB03



예제: COMPANY 데이터베이스

요구사항 Requirements

1. 부서 DEPARTMENT
    a. 기업은 여러 가지 부서로 조직되어 있음
    b. 고유한 이름, 번호, 위치, 담당자, 부서 담당 시작 날짜의 정보를 가짐
    c. 부서는 프로젝트를 관리
2. 프로젝트 PROJECT
    고유한 이름, 번호, 위치의 정보를 가짐
3. 사원 EMPLOYEE
    a. 이름, SSN(주민번호), 주소, 연봉, 성별, 생년월일의 정보를 가짐
    b. 하나의 부서에 배정됨
    c. 여러 개의 프로젝트에 일할 수 있음
    d. 일주일에 몇 시간 동안 각 프로젝트에 일하는지 기록
    e. 사원의 상사 정보 기록
    f. 사원의 부양 가족 기록
        부양 가족: 성, 성별, 생년월일, 관계



Entity 엔티티 (개체)
실세계에 독립적으로 존재하는 어떤 것 Thing in real world with independent existence
e.g. 사람, 차, 집, 학생 등

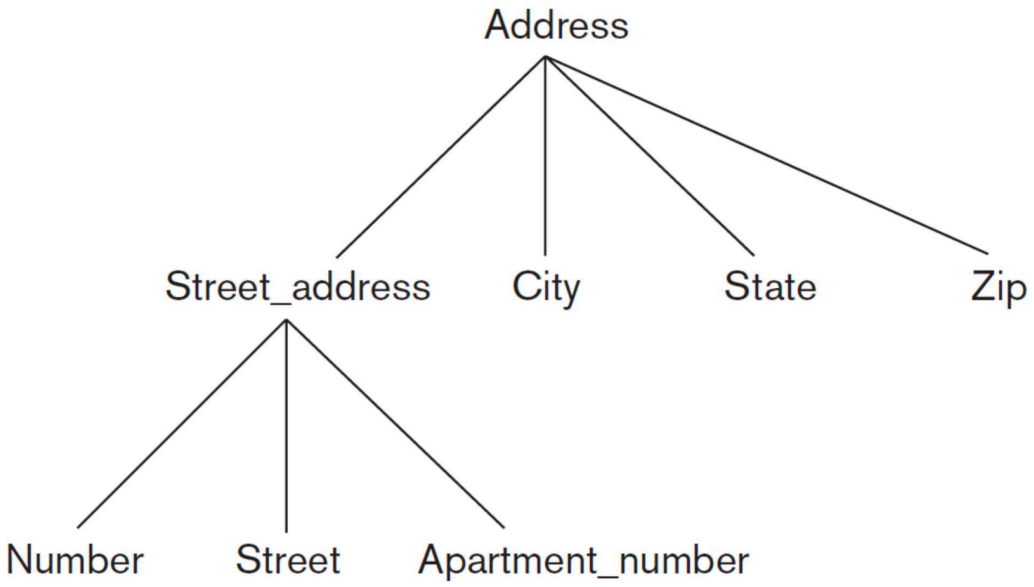
Attributes 특성
엔티티를 설명하는 특성
e.g. 이름, 나이, 주소, 직업 등

특성의 종류

카테고리 1

- Simple (atomic) attributes: 더 이상 쪼갤 수 없는 특성

- Composite attributes: 더 작은 단위로 쪼갤 수 있는 특성. 계층 구조를 형성할 수 있다.



카테고리 2

- Single-valued attributes: 특정 엔티티에 대해 하나의 값만 가질 수 있다.
- Multivalued attributes (set attributes): 특정 엔티티에 대해 여러 개의 값을 가질 수 있다.

카테고리 3

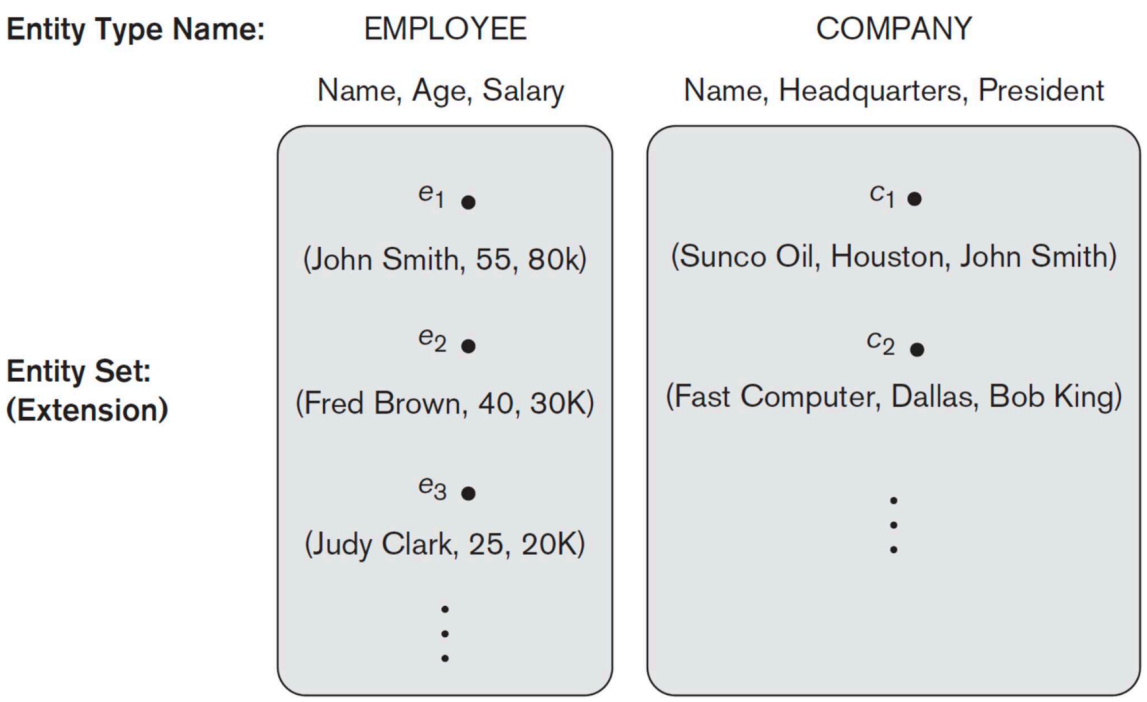
- Stored attributes: 실제로 저장되어 있는 특성
- Derived attributes: 실제로 저장되어 있지는 않지만 다른 특성을 통해 생성할 수 있는 특성

  e.g. 나이 → 생년월일을 통해 구할 수 있음

Entity Type 엔티티 유형

A collection of entities that have a same set of attributes 같은 특성의 집합을 공유하는 엔티티의 집합

- 엔티티의 집합에 대한 스키마를 나타낸다.
- 엔티티 유형은 이름과 특성으로 설명된다.



Domains 도메인
특정 엔티티의 특성에 할당될 수 있는 값의 집합(범위)
e.g. EMPLOYEE 엔티티의 AGE 특성의 도메인: 20 ~ 70

Key Attribute

각 엔티티에 대해 서로 다른, 고유한 값을 갖는 특성.

→ 엔티티를 식별하는데 사용된다.

e.g. STUDENT 엔티티의 학번 특성

- 여러 개의 특성이 하나의 key를 형성할 수 있다.
- 각 엔티티 유형은 2개 이상의 key를 가질 수 있다.
- 특정 key의 고유성은 특정 엔티티의 집합이 아닌, 모든 엔티티의 집합에 적용될 수 있어야 한다.

📌 우리 반에 이름이 같은 학생이 하나도 없더라도, 이름을 key attribute로 사용할 수는 없다. Key attribute는 데이터베이스 설계 초기에 정하는 것. '하다 보니 고유해서 key attribute다!'라고 할 수는 없다는 이야기.

COMPANY 데이터베이스의 초기 개념적 설계

DEPARTMENT 부서

이름, 번호, {위치}, 담당자, 담당 시작 날짜

PROJECT 프로젝트

이름, 번호, 위치, 담당 부서

EMPLOYEE 사원

성명(성, 미들네임, 이름), SSN, 성별, 주소, 연봉, 생년월일, 부서, 상사, {근무 시간(프로젝트, 시간)}

DEPENDENT 피부양자

사원, 피부양자 성명, 성별, 생년월일, 관계

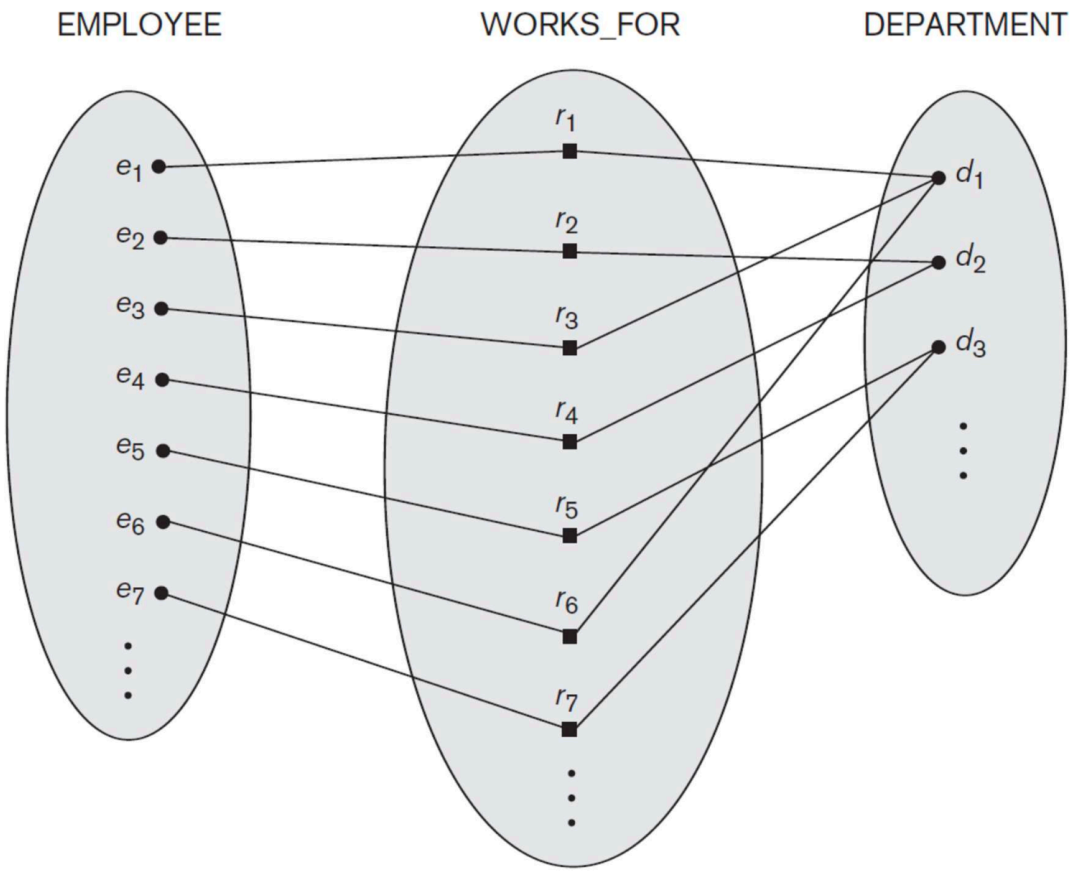
> {}: multivalued attribute, (): composite attribute

관계 Relationship

Relationship instances 관계 인스턴스: 엔티티 간의 관계를 나타낸다.

Relationship type 관계 유형: 관계 인스턴스의 집합.

예제: WORKS_FOR 관계



r1, r2, r3 → 관계 인스턴스

WORKS_FOR → 관계 유형

Relationship Degree 관계 정도

관계에 참여하는 엔티티 유형의 수.

Binary Relationship

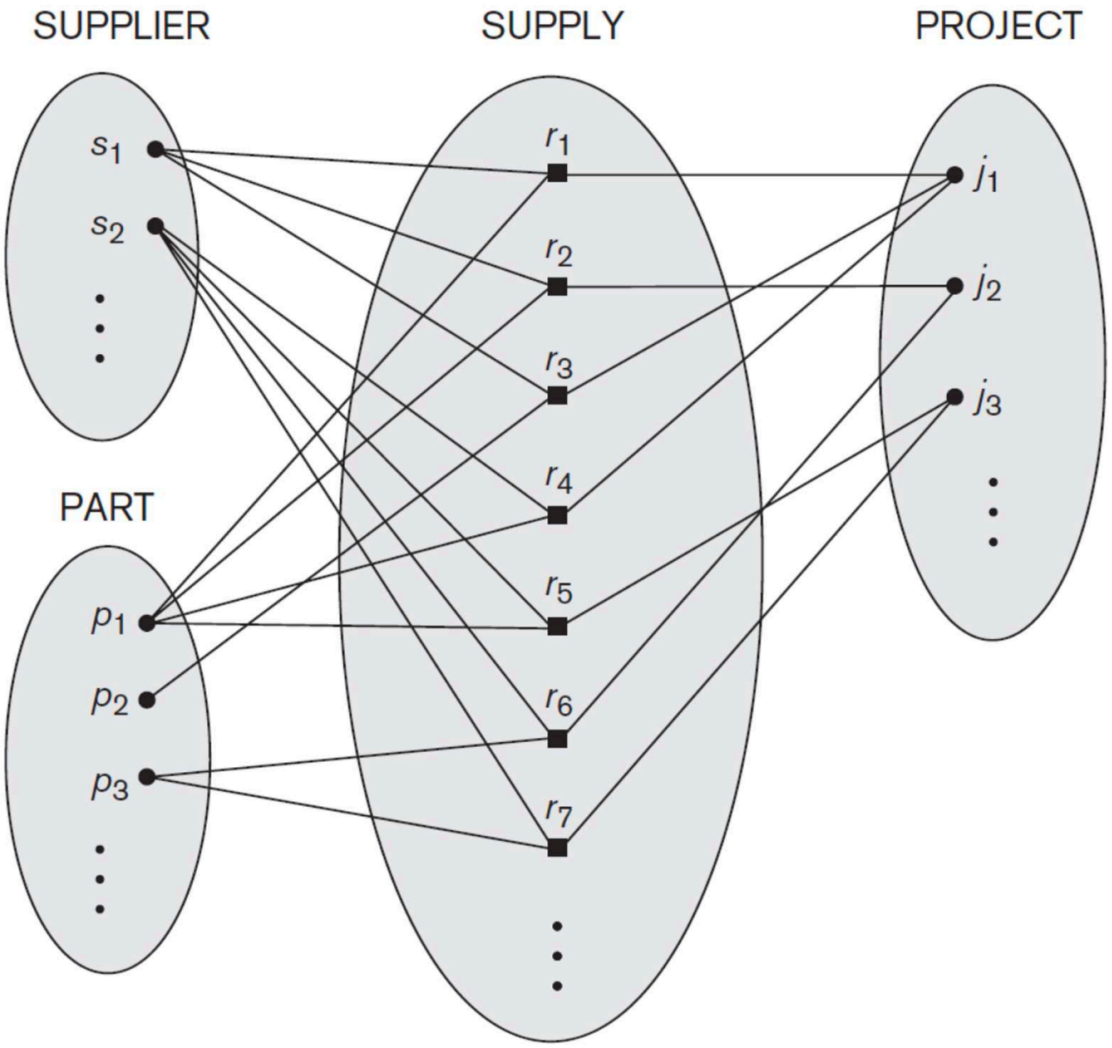
관계 정도가 2인 관계 유형. 관계에 참여하는 엔티티 유형이 2개다.

e.g. WORKS_FOR 유형은 EMPLOYEE와 DEPARTMENT 엔티티 유형, 총 2개가 참여한다.

Ternary Relationship

관계 정도가 3인 관계 유형.

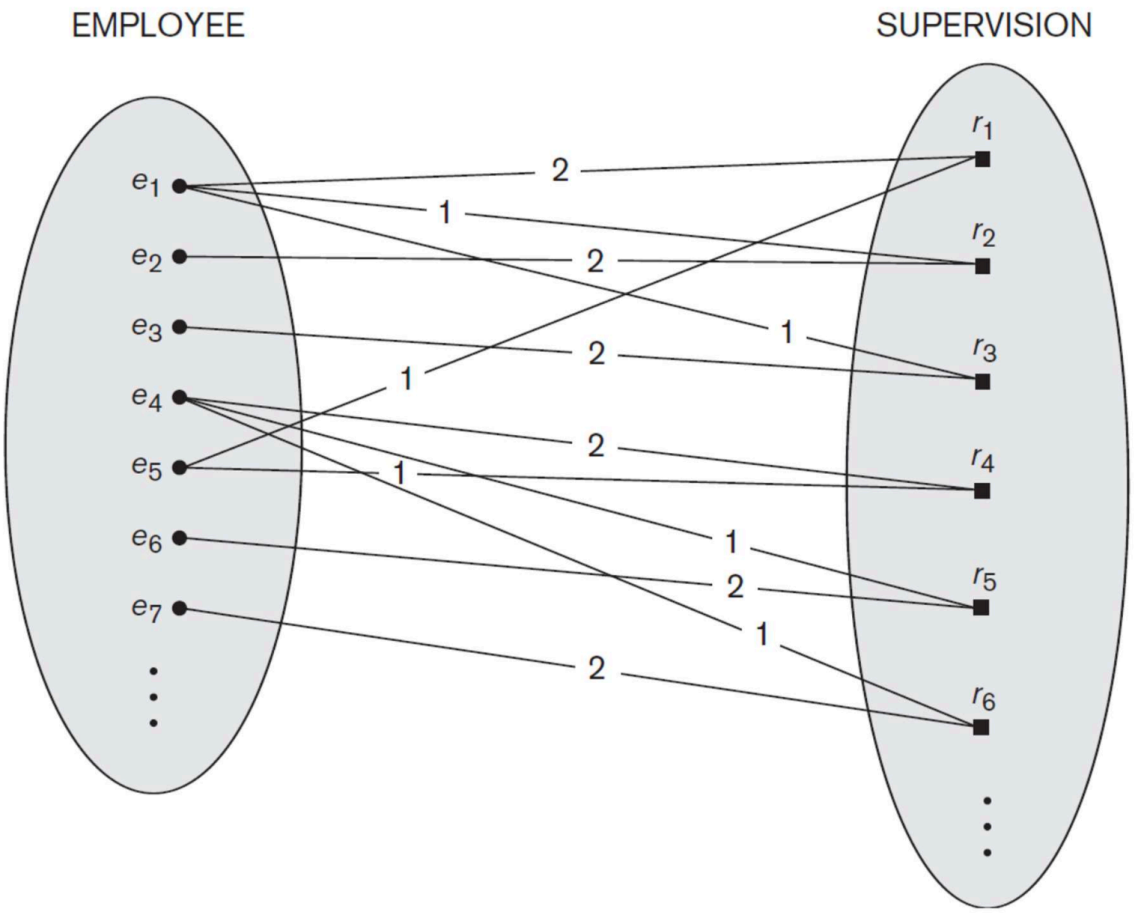
e.g. SUPPLY 관계 유형은 SUPPLIER, PART, PROJECT 총 3개가 참여한다.



Recursive Relationships 재귀적 관계

Role Name 역할명
참여하는 엔티티가 관계 인스턴스에서 어떤 역할을 하는지 나타낸다.

Recursive Relationship Type 재귀적 관계 유형
같은 엔티티 유형이 특정 관계 유형에 두 번 이상 다른 역할로 참여하는 경우.



1번은 상사, 2번은 부하 직원으로써 관계에 참여한다.

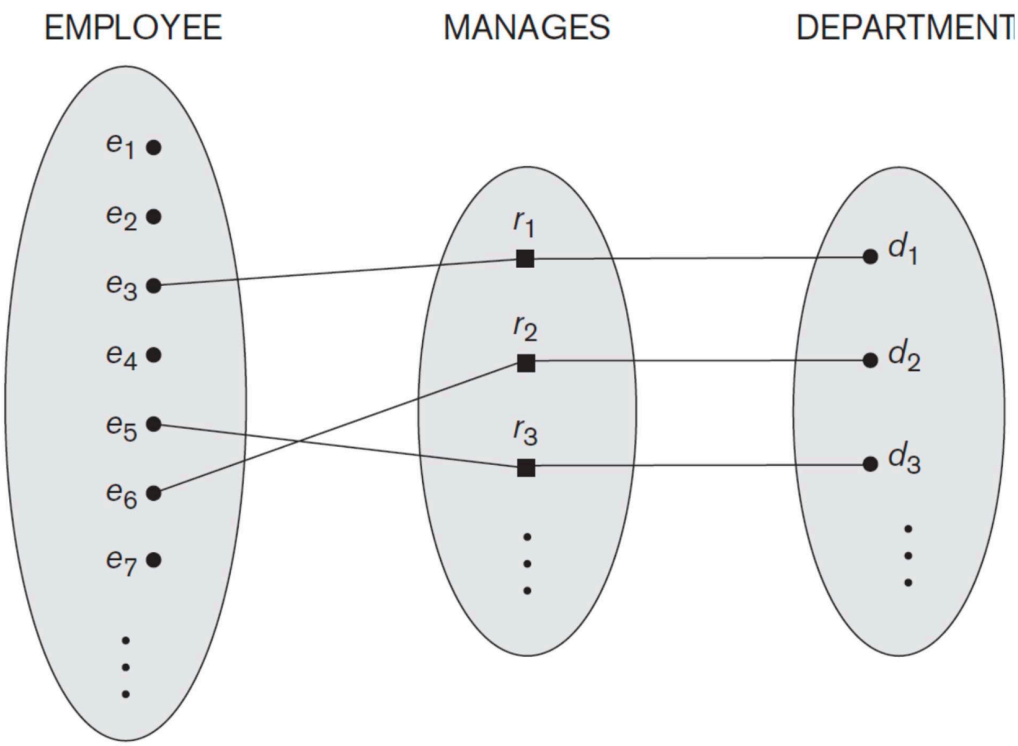
▼ DB04

Constraints

1. Cardinality ratio: 엔티티가 참여할 수 있는 관계 인스턴스의 최대 개수

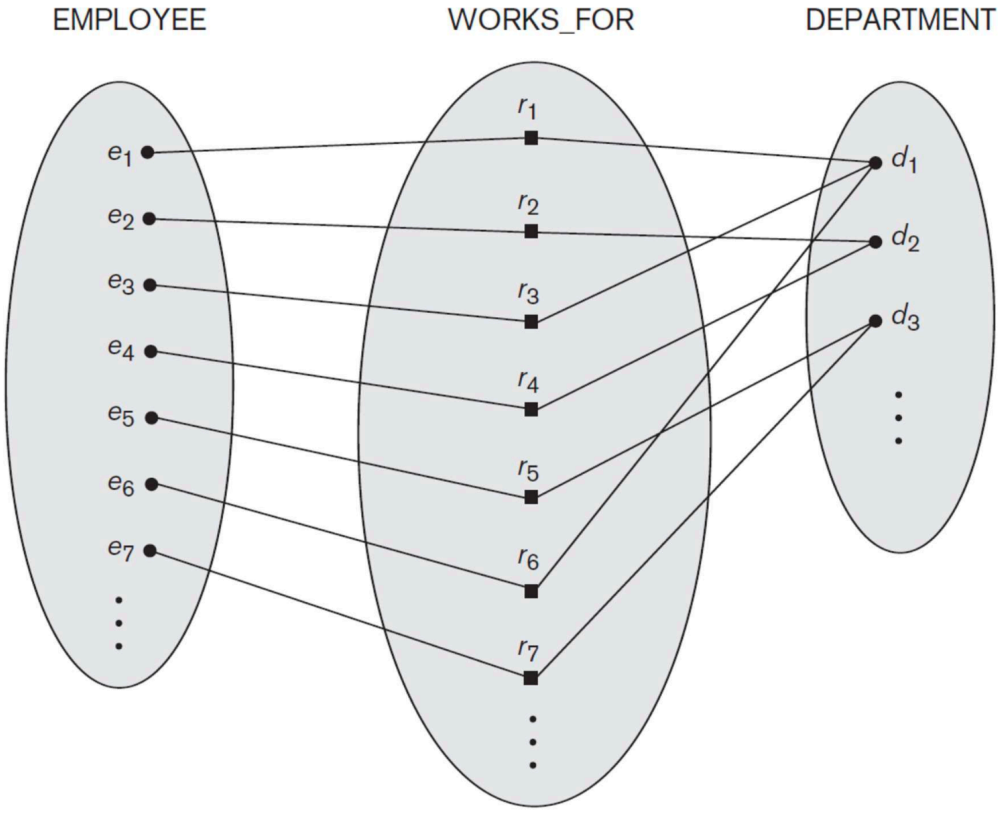
   Binary relationship에는 1:1, 1:N, M:N 총 3가지의 cardinality ratio가 존재한다.

   a. 1:1



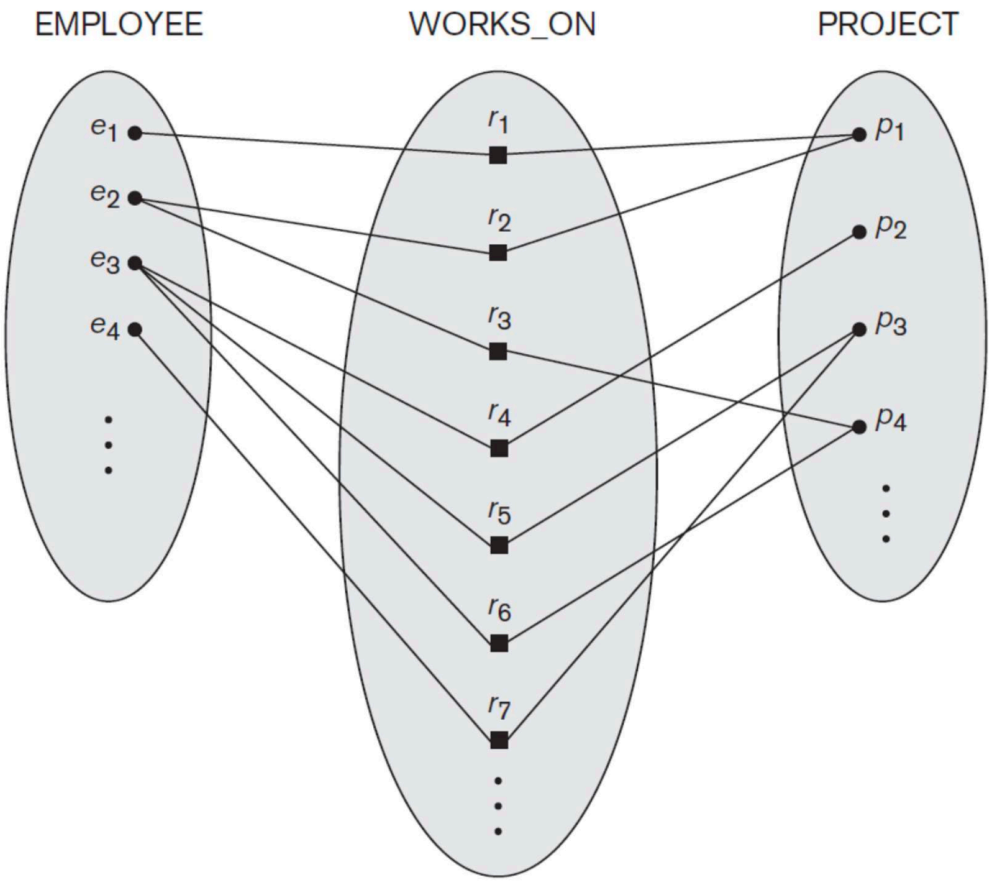
1개의 EMPLOYEE 엔티티는 1개의 DEPARTMENT 엔티티에 대해 1개의 MANAGES 관계를 가질 수 있다.

b. 1:N



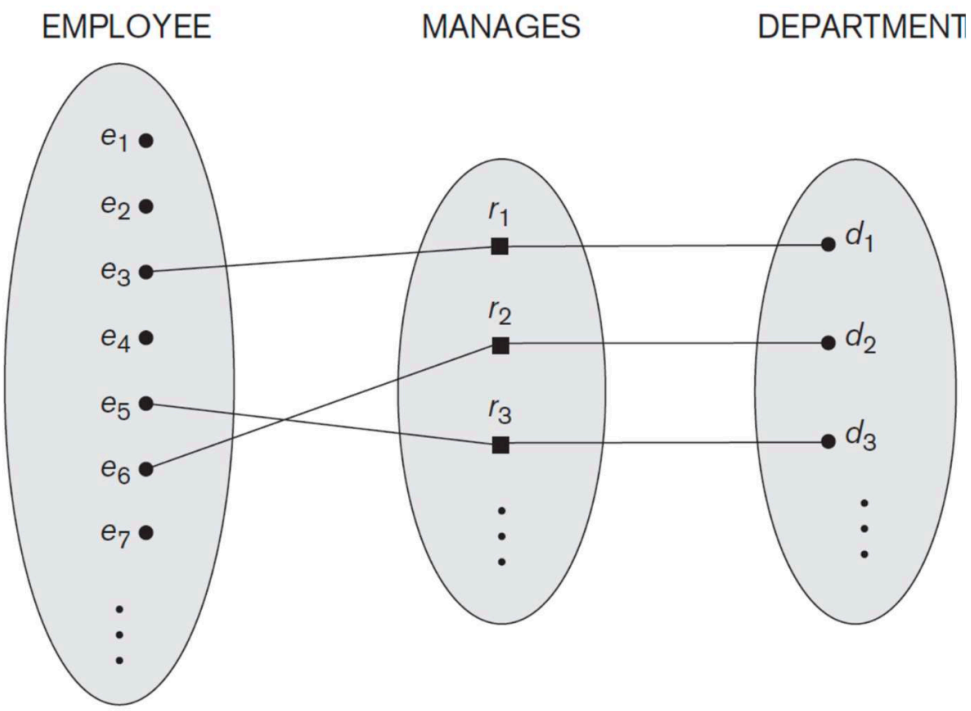
1쪽의 DEPARTMENT 엔티티는 N쪽의 여러 EMPLOYEE 엔티티와 관계를 가질 수 있다. d1은 e1, e3, e6 총 3개의 EMPLOYEE 엔티티와 관계를 가진다.

c. M:N



여러 개의 EMPLOYEE 엔티티가 여러 개의 PROJECT 엔티티에 대해 WORKS_ON 관계를 가질 수 있다.

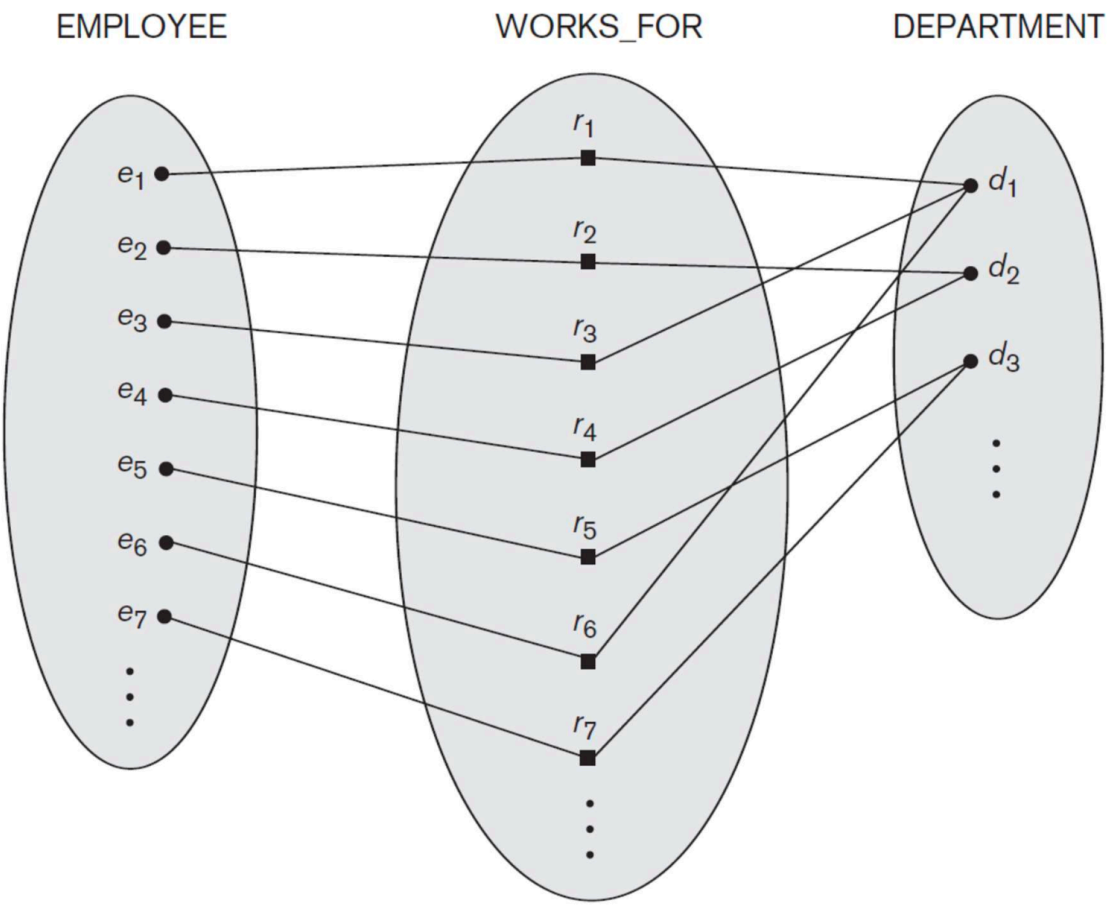
2. Participation constraint: 엔티티의 존재 여부가 관계 참여 여부에 따라 결정된다.
   a. Total: 모든 엔티티는 최소 1개의 관계 인스턴스에 참여해야 한다.
   b. Partial: 일부 엔티티만 관계 인스턴스에 참여한다.



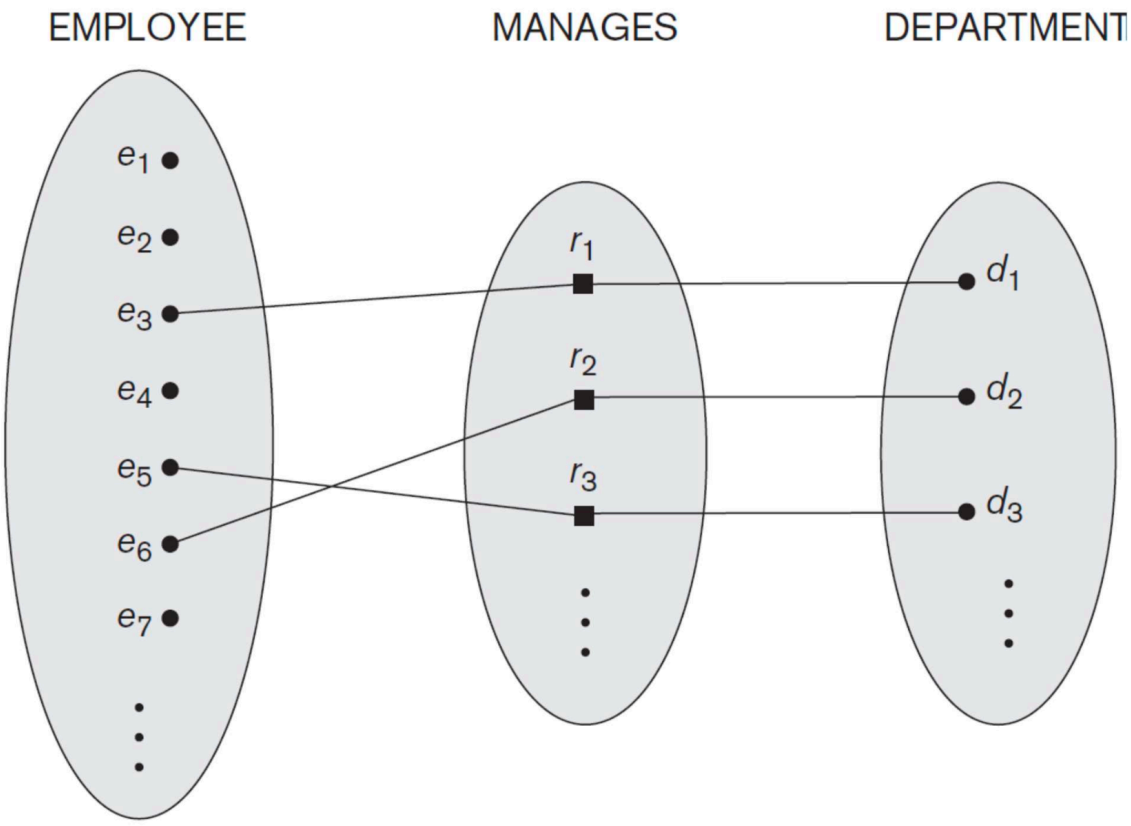
이 예제에서 DEPARTMENT는 total, EMPLOYEE는 partial participation에 해당한다.

Attributes 속성

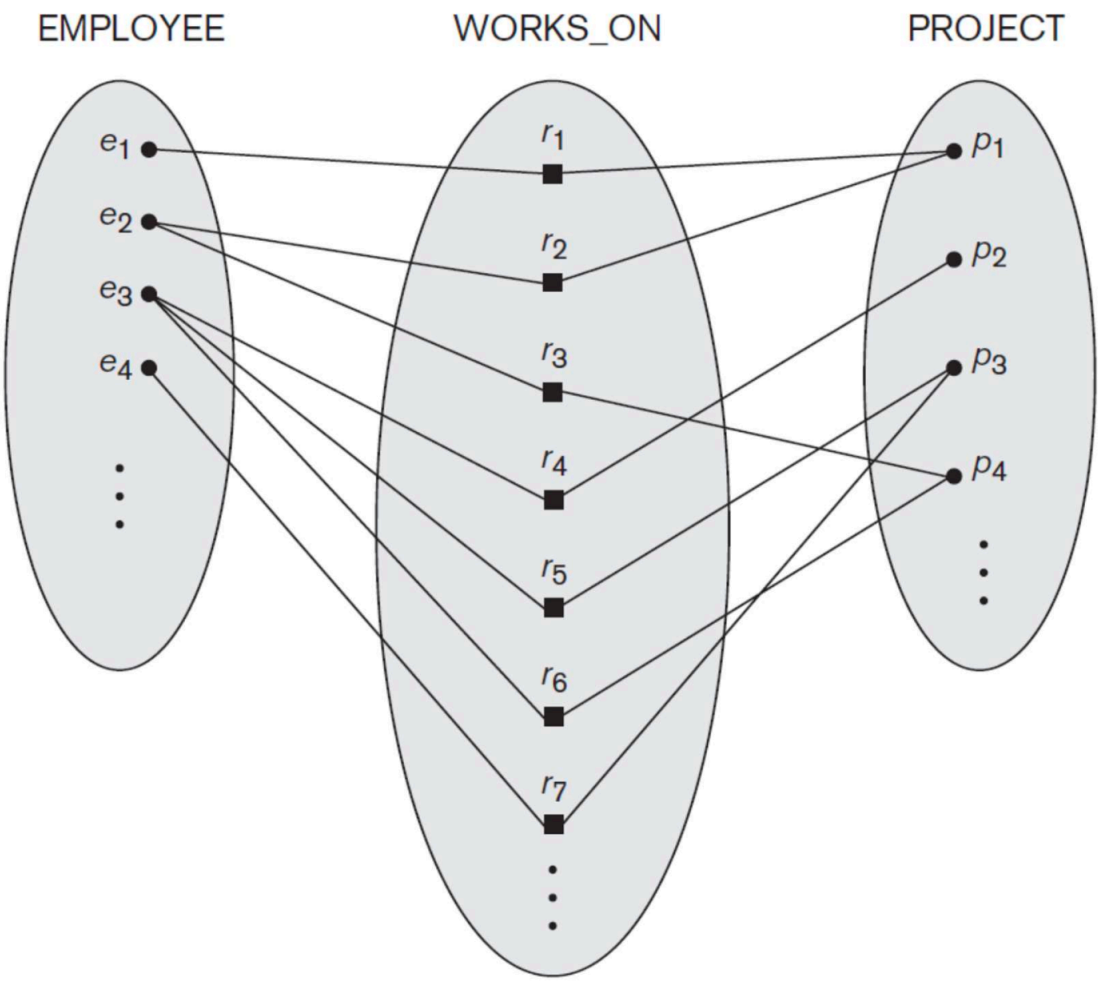
관계 유형은 attributes 속성을 가질 수 있다. 이는 참여하는 엔티티의 조합에 따라 결정되고, 각 관계 인스턴스의 특징을 나타낸다. 하나의 엔티티 유형 전체에 대해 특성을 정의하기 어려울 때 사용된다.



StartDate라는 속성을 사용하여 EMPLOYEE 엔티티가 DEPARTMENT 엔티티에서 일하기 시작한 날짜를 기록한다. 위의 경우 1:N 관계이므로 사실 관계 유형에 대한 속성보다는 그냥 N쪽인 EMPLOYEE 엔티티 유형에 대한 속성으로 정의해도 무방하다.



MgrStartDate로 EMPLOYEE 엔티티가 DEPARTMENT 엔티티를 관리하기 시작한 날짜를 기록한다. 마찬가지로 1:1 관계에서도 EMPLOYEE 또는 DEPARTMENT 엔티티 유형에 대한 속성으로 정의해도 무방하다.



반면 M:N 관계에서는 각 관계에 대한 속성을 정의할 필요성이 생긴다. 위의 예제에서 Hours라는 속성을 사용하여 EMPLOYEE 엔티티가 PROJECT 엔티티에 일한 시간을 기록한다.

그러나 이 경우 EMPLOYEE 엔티티 유형에 Hours를 정의하면 모호하다. 예를 들어 e2의 Hours가 p1에 일한 시간인지, 아니면 p4에 일한 시간인지 정확히 알 수 없다. 마찬가지로 PROJECT 엔티티 유형에 Hours를 정의하더라도 p1의 Hours가 e1이 일한 시간인지, 아니면 e2가 일한 시간인지 알 수 없다.

따라서 특정 관계 엔티티 r1, r2, r3...에 대한 속성으로 정의해야 어느 EMPLOYEE 엔티티가 어느 PROJECT 엔티티에 일한 시간인지 정확히 알 수 있다.

Weak Entity Types

> **Weak Entity Types**
> 자신의 key attribute를 가지지 않는다.

e.g. DEPENDENT 엔티티 유형: DepedentName, BirthDate, Sex, Relationship

- A's son (Gildong, 1/1/14, M, son)
- B's son (Gildong, 1/1/14, M, son)

Identifying owner

부모가 되는 다른 엔티티 유형 identifying owner을 통해 식별 가능하다.

e.g. EMPLOYEE 엔티티 유형으로 DEPENDENT 엔티티를 식별

- A's son (Gildong, 1/1/14, M, son): linked to A
- B's son (Gildong, 1/1/14, M, son): linked to B

Identifying Relationship Type 식별 관계 유형

Weak entity type과 identifying owner를 연결하는 관계 유형이다. Weak entity type은 자신의 key attribute가 없기 때문에 항상 total participation constraint를 가진다.

e.g. DEPENDENTS_OF 관계 유형으로 DEPENDENT와 EMPLOYEE 간의 관계 정의

Partial Key

같은 부모 엔티티를 갖는 weak entity 간을 구분하는 역할. 식별 관계와 합쳐 weak entity type을 식별한다.

e.g. DEPENDENT 엔티티 유형의 DependentName 속성을 통해 DEPENDENT 엔티티 간을 식별

COMPANY 데이터베이스 예제

계속해서 COMPANY 데이터베이스 예제를 이어가보자. 먼저 초기 개념적 설계 initial conceptual design를 살펴보자.

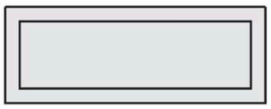
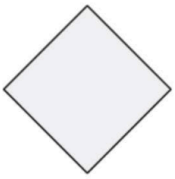
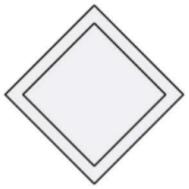
- DEPARTMENT: Name, Number, {Locations}, Manager, ManagerStartDate
- PROJECT: Name, Number, Location, ControllingDepartment
- EMPLOYEE: Name(FName, MInit, LName), SSN, Sex, Address, Salary, BirthDate, Department, Supervisor, {WorksOn(Project, Hours)}
- DEPENDENT: Employee, DependentName, Sex, BirthDate, Relationship

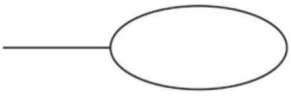
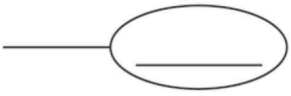
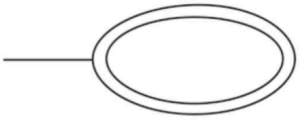
이제 관계 유형을 추가해볼 것이다.

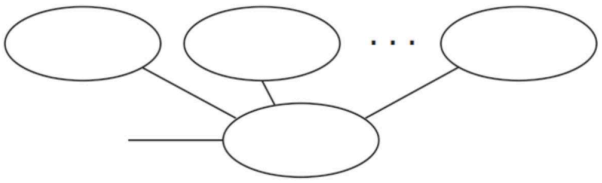
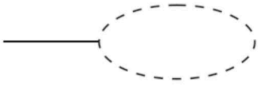
- MANAGES
  - EMPLOYEE와 DEPARTMENT 간의 1:1 관계
  - EMPLOYEE는 부분 참여
  - DEPARTMENT는 전체 참여
  - **MgrStartDate** attribute을 가짐
- WORKS_FOR
  - EMPLOYEE와 DEPARTMENT 간의 1:N 관계
  - 두 엔티티 모두 전체 참여
- CONTROLS
  - DEPARTMENT와 PROJECT 간의 1:N 관계
  - PROJECT는 전체 참여
  - DEPARTMENT는 부분 참여
- SUPERVISION
  - EMPLOYEE (supervisor)와 EMPLOYEE (supervisee) 간의 1:N 관계
  - 두 엔티티 모두 전체 참여
- WORKS_ON
  - EMPLOYEE와 PROJECT 간의 M:N 관계
  - 두 엔티티 모두 전체 참여
  - **Hours** attribute을 가짐
- DEPENDENTS_OF
  - EMPLOYEE와 DEPENDENT 간의 1:N 관계
  - DEPENDENT의 식별 관계
  - EMPLOYEE는 부분 참여
  - DEPENDENT는 전체 참여

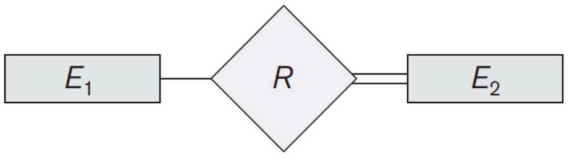
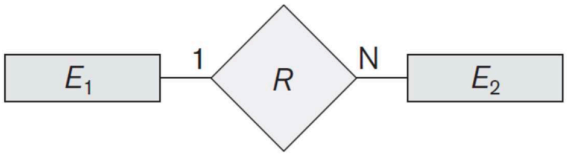
Entity-Relationship Diagram

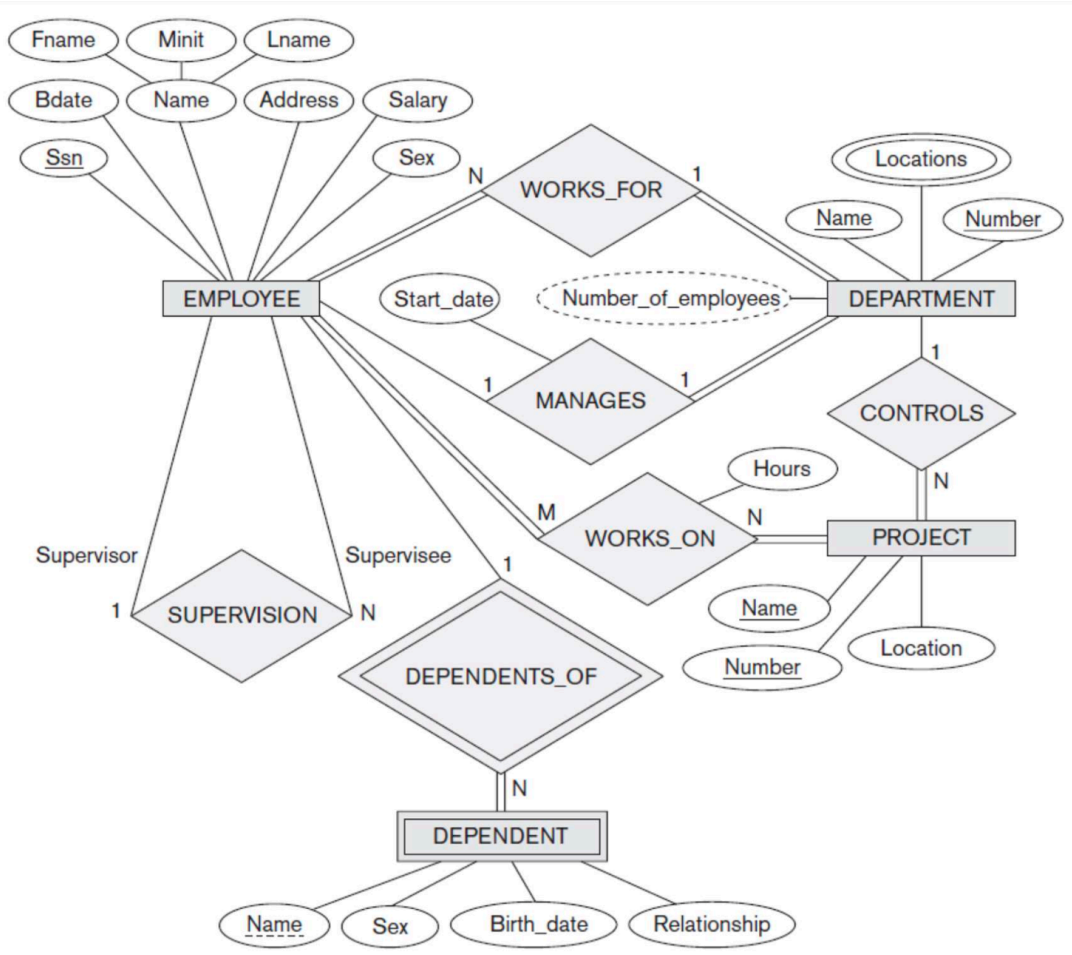
ER 모델을 통해 만든 개념적 설계를 나타내는 다이어그램이다.

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship

Symbol	Meaning
	Attribute
	Key Attribute
	Multivalued Attribute

Symbol	Meaning
	Composite Attribute
	Derived Attribute

Symbol	Meaning
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for E_1:E_2 in R



▼ DB05

Relational Model 관계형 모델

Database 데이터베이스
Collection of relations (or tables)

Relation 관계 (Tables)
Set of tuples (or rows)

Tuple 튜플

- A fact that typically corresponds to a real-world entity or relationship 실세계의 개체/관계에 대응되는 사실

- Attributes의 집합으로 구성됨

Attributes 속성

Defines a property of each object corresponding to a tuple 튜플에 대응되는 객체의 속성을 정의한다.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

PREREQUISITE

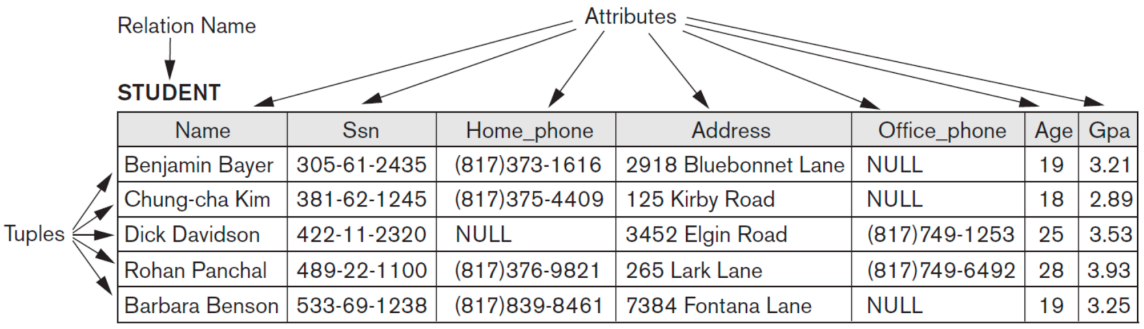
Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A



Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

Domain D 범위

A set of atomic values 더 이상 쪼갤 수 없는 값들의 집합

e.g. domain of Age attribute: 16 ~ 80

Attribute A_i 속성

범위 D 내에서 어떤 역할을 하는지 나타낸다.

> $\mathrm{dom}(A_i)$: 속성 A_i의 범위

e.g. 범위 dom(Employee_ages) (=D)

Relation Schema R 관계 스키마

관계명 R과 속성의 집합 $A_1, A_2, ..., A_n$으로 구성된다.

> $R(A_1, A_2, ..., A_n)$

e.g. STUDENT(Name, SSN, HomePhone, Address, OfficePhone, Age, GPA)

Degree

Relation schema에 포함되는 속성의 수

Tuple t of relation R

n-tuple: 관계에 들어있는 속성이 n개 존재한다.

> n개의 값의 정렬된 리스트 형태로 표기: $t =< v_1, v_2, ..., v_n >$

각 값 v_i (1 ≤ *i* ≤ *n*)는 $\mathrm{dom}(A_i)$의 원소거나 NULL 값을 가진다.

Relation Instance $r(R)$

튜플의 집합: $r(R) = \{t_1, t_2, ..., t_m\}$
R을 정의하는 범위의 곱집합의 부분집합이다: $r(R) \subseteq (\mathrm{dom}(A_1) \times \mathrm{dom}(A_2) \times ... \times \mathrm{dom}(A_n))$

Characteristics of Relations

1. 튜플의 순서?

   Relation은 튜플의 집합으로 정의되므로 튜플의 순서는 중요하지 않다.

2. Attributes의 순서?

   N-튜플은 n개의 값의 정렬된 리스트로 정의되므로 속성과 그 값의 순서는 중요하지 않다.

3. 튜플에 저장되어 있는 값
    a. 튜플의 각 값들은 atomic 더 이상 쪼갤 수 없다.

        Composite and multivalued attributes를 허용하지 않는다 → First normal form

    b. NULLs: unknown 또는 not applicable한 값을 나타낸다.

Notation 표기법

$t[A_i]$: 튜플 t의 속성 A_i가 갖는 값 v_i를 가리킴

$t[A_u, A_w, ..., A_z]$: 부분 튜플의 값들 $< v_u, v_w, ..., v_z >$을 나타낸다.

Constraints 제약 조건

- Relation schema의 실제 값에 대한 제한사항
- 해당 relation의 모든 인스턴스는 이 제약 조건을 따라야 한다.
- Mini-world 소세계의 규칙 rules를 통해 결정된다.

관계형 모델의 주요 제약 조건

1. Key Constraints 키 제약 조건

Superkey SK 슈퍼키
R의 속성 SK의 집합. 관계 인스턴스 $r(R)$의 서로 다른 두 튜플이 SK에 해당하는 속성에 대해 **같은 값을 가져서는 안된다**는 제약 조건이다.
t_1과 t_2 $(t_1 \neq t_2)$에 대해 $t_1[\mathrm{SK}] \neq t_2[\mathrm{SK}]$
Uniqueness constraint 고유성 제약 조건이라고 부른다.

Candidate key K 후보 키

- R의 슈퍼키
- Minimal 최소 슈퍼키: K에서 아무 속성 A를 제거하면 더 이상 슈퍼키가 아니게 되는 슈퍼키

- 관계 스키마는 하나 이상의 후보 키를 가질 수 있다.

CAR

License_number	Engine_serial_number	Make	Model	Year
Texas ABC-739	A69352	Ford	Mustang	02
Florida TVP-347	B43696	Oldsmobile	Cutlass	05
New York MPO-22	X83554	Oldsmobile	Delta	01
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	04
Texas RSK-629	U028365	Jaguar	XJS	04

위의 예제에서 6개의 튜플 모두 서로 다른 값을 가지고 있기 때문에 모든 속성의 집합이 슈퍼키가 될 수 있다. 그러나 이 중 아무 속성 하나를 빼더라도 여전히 슈퍼키이므로 최소 슈퍼키가 아니기 때문에 후보 키는 아니다.

예를 들어 {License_number}, 이 속성 하나만 남겨놨다고 가정해보자. 이 집합은 슈퍼키이면서 후보 키다. 만약 License_number 속성을 제거하면 더 이상 슈퍼키가 아니기 때문이다.

e.g. STUDENT 관계 스키마에서 {SSN, Name, Age} 는 슈퍼키지만 후보 키는 아니다. {SSN}은 슈퍼키면서 후보 키다.

Primary key PK 기본 키
후보 키 중 하나를 데이터베이스 설계자가 선택한다. 이 키는 밑줄로 표시한다.
e.g. DEPARTMENT 관계 스키마의 DepartmentID, DepartmentName

Key constraint 키 제약 조건
어떤 관계(테이블)는 반드시 후보 키를 가져야 한다는 조건

Relational Database Schema 관계형 데이터베이스 스키마

관계형 데이터베이스 스키마는 다음으로 이루어져 있다:

- Set of relation schemas 데이터베이스 내의 관계 스키마들의 집합: $S = \{R_1, R_2, ..., R_m\}$
- Set of integrity constraints IC 무결성 제약 조건의 집합

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

Relational database instance 관계형 데이터베이스 인스턴스

- 데이터베이스 내의 관계 인스턴스들의 집합: $\{r_1, r_2, ..., r_m\}$
- r_i: IC에 정의된 무결성 제약 조건을 만족하는 R_i의 인스턴스

2. Entity Integrity Constraints 엔티티 무결성 제약 조건

각 관계 스키마 R의 기본 키에 대한 제약 조건. 기본 키 Primary key는 NULL 값을 가질 수 없다는 제약 조건이다.

- $r(R)$에 속하는 모든 튜플 t에 대해 $t[\mathrm{PK}] \neq$ NULL
- 기본 키의 값은 각 튜플을 식별하는데 사용되기 때문이다.

3. Referential Integrity Constraints 참조 무결성 제약 조건

두 테이블에 관련된 제약 조건. 키 제약 조건과 엔티티 제약 조건은 하나의 테이블에 대한 제약 조건이었다면, 참조 무결성 제약 조건은 **두 테이블의 튜플 간의 관계의 일관성**을 정의한다.

> 하나의 테이블에 속하는 튜플이 다른 테이블에 존재하는 튜플을 참조해야 한다. 없는 튜플을 참조해서는 안된다.

e.g. EMPLOYEE의 DNO 속성은 DEPARTMENT 테이블의 DNumber 속성을 참조할 때 반드시 존재하는 값을 참조해야 한다.

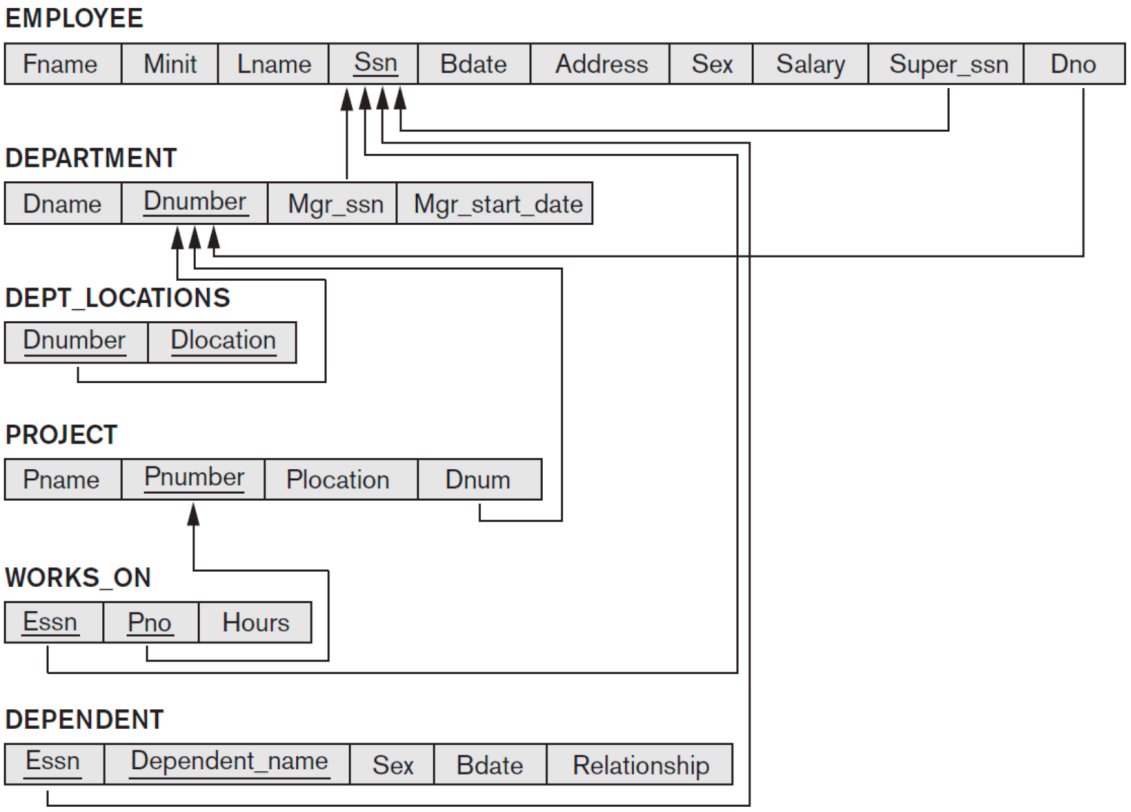
Foreign Key FK 외래 키

외래 키는 참조하는 관계 스키마 R_1과 참조의 대상이 되는 관계 스키마 R_2에 대해 다음의 조건들을 만족한다:

1. R_1의 FK와 R_2의 PK는 같은 범위를 갖는다.
2. $r_1(R_1)$ 인스턴스의 임의의 튜플 t_1의 FK는 $r_2(R_2)$ 인스턴스의 임의의 튜플 t_2의 PK의 값을 가지거나 NULL 값을 가진다.

$t_1[\mathrm{FK}] = t_2[\mathrm{PK}]$ (or NULL): t_1 refers to t_2

참조 무결성 제약 조건은 이 외래 키를 정의하는 제약 조건으로, 하나의 FK에서 참조하는 PK로 화살표로 나타낸다.



▼ DB06

ER-to-Relational Model Mapping

ER 모델을 관계형 모델에 매핑하는 작업을 해보자.

Regular Entity Types

- 일반적인 엔티티 유형은 관계형 모델의 **entity relation 엔티티 관계**로 매핑
- 엔티티 유형의 simple attribute 기본 속성을 그대로 매핑
- 엔티티 유형의 key attributes 중 하나를 선택하여 기본 키로 사용

Weak Entity Types

- 약한 엔티티 유형도 관계형 모델의 relation 관계로 매핑
- 약한 엔티티 유형의 simple attributes는 그대로 매핑하고, 약한 엔티티 유형의 부모 엔티티 owner의 기본 키 PK를 관계형 모델의 외래 키 FK로 사용

- 엔티티 유형의 key attributes 중 하나를 선택하여 외래 키와 조합하여 기본 키로 사용

Multivalued Attributes

- Multivalued attribute를 관계형 모델의 하나의 관계로 매핑
- Multivalued attribute는 그대로 관계의 속성으로 매핑하고, 기존에 해당 속성의 엔티티의 key attribute를 외래 키로 추가
- 모든 속성의 조합 Multivalued attributes + FK를 기본 키 PK로 사용

Composite Attributes

- 각각의 composite attribute 하나하나를 새로운 relation으로 매핑
- Composite attributes의 simple attributes들은 그대로 속성으로 매핑하고, 엔티티 유형 관계의 key attributes를 외래 키로 사용
- 이 속성의 외래 키를 관계형 모델의 기본 키로 사용
- 또 다른 방식: 모든 simple component attributes를 대응되는 관계로 매핑

Binary 1:1

- 관계에 참여하는 양쪽 엔티티 식별
- 이 관계 중 하나를 선택하고 나머지 관계의 PK를 FK로 사용

  → 이때 Total participation 전체 참여하는 엔티티 유형을 선택하는 것이 좋음: 나머지의 PK를 Total의 FK로 사용
- 기존 1:1 관계의 기본 속성들은 선택한 관계의 속성으로 추가

Binary 1:N

- 관계에 참여하는 N쪽 엔티티 식별
- 반대쪽(1쪽) 엔티티의 PK를 N쪽 엔티티의 FK로 사용
- 기존 1:N 관계의 기본 속성들은 N쪽 관계의 속성으로 추가

Binary M:N

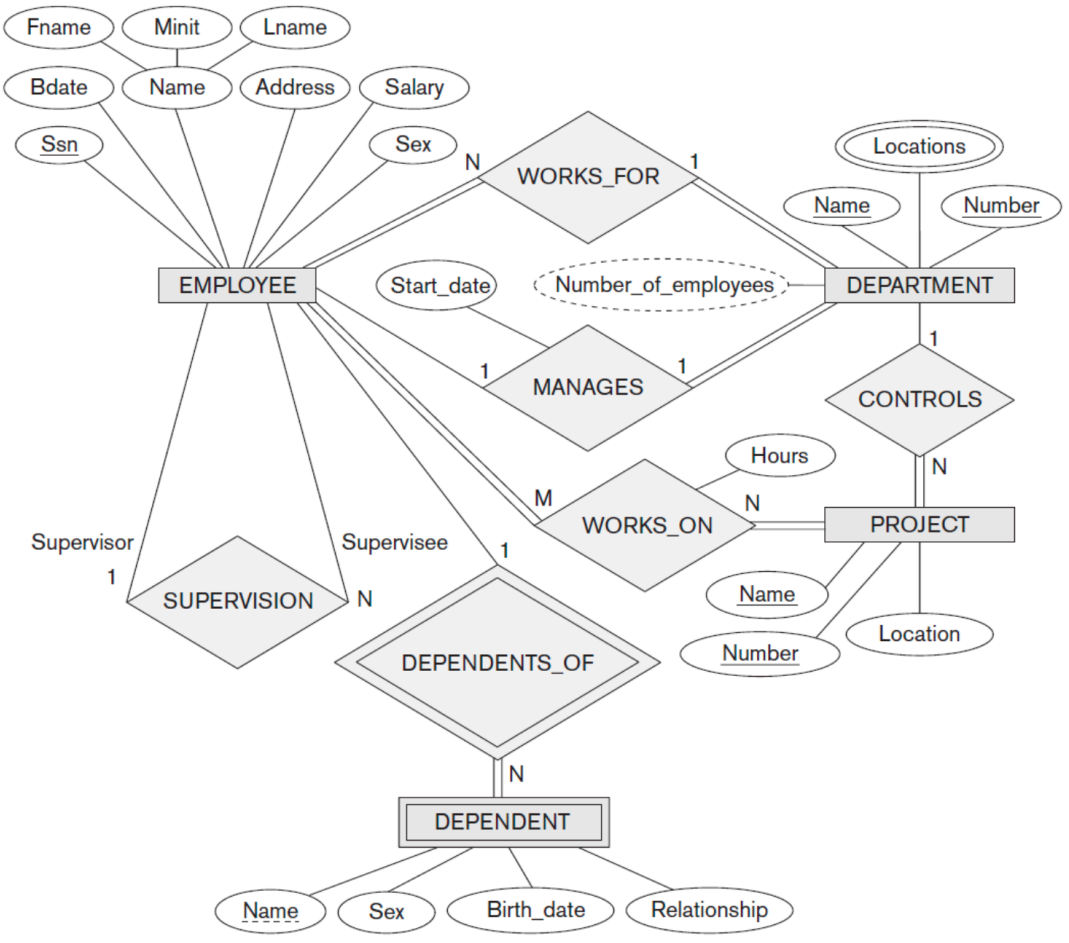
- 하나의 독립적인 관계로 정의

- 참여하는 양쪽 엔티티 유형의 PK를 FK로 사용하고, 기본 속성은 그대로 사용
- FK를 모두 합쳐서 PK로 사용

N-ary

- 각각의 N-ary 관계 유형을 relations로 정의
- 참여하는 엔티티 유형의 PK를 FK로 사용하고, 기본 속성은 그대로 사용
- FK를 모두 합쳐서 PK로 사용

예제: COMPANY 데이터베이스



Regular Entity Types

1. EMPLOYEE
    a. 새 관계 EMPLOYEE 생성

    b. Simple attributes 추가: **Ssn**, **Bdate**, **Address**, **Sex**, **Salary**
    c. **Ssn**을 PK로 사용
2. DEPARTMENT
    a. 새 관계 DEPARTMENT 생성
    b. Simple attributes 추가: **Name** as **Dname**, **Number** as **Dnumber**
    c. **Dnumber**를 PK로 사용
3. PROJECT
    a. 새 관계 PROJECT 생성
    b. Simple attributes 추가: **Name** as **Pname**, **Number** as **Pnumber**, **Location** as **Plocation**
    c. **Pnumber**를 PK로 사용

Weak Entity Types

DEPENDENT

- 새 관계 DEPENDENT 생성
- Simple attributes 추가: **Name** as **Dependent_name**, **Sex**, **Birth_date** as **Bdate**, **Relationship**
- Owner 부모 엔티티(EMPLOYEE)의 PK를 속성으로 추가: **Ssn** as **Essn**
- **Essn** + **Depedent_name**을 PK로 사용

Multivalued Attributes

Locations in DEPARTMENT

- 새 관계 DEPT_LOCATIONS 생성
- DEPARTMENT의 PK를 FK로 추가: **Dnumber**
- Multivalued attribute 추가: **Locations** as **Dlocation**
- **Dnumber** + **Dlocation**을 PK로 사용

Composite Attributes

EMPLOYEE의 **Name**

→ **Name**의 simple attribute들을 추가: **Fname**, **Minit**, **Lname**

Binary 1:1

MANAGES

- 참여하는 관계: EMPLOYEE & DEPARTMENT
- 이 중 전체 참여는 DEPARTMENT
- 따라서 EMPLOYEE의 PK를 DEPARTMENT의 FK로 추가: **Ssn** as **Mgr_ssn**

> EMPLOYEE 중 일부만이 DEPARTMENT를 MANAGE하기 때문에 이렇게 매핑하는 것이 상식적이다. DEPARTMENT 쪽에 팀장의 Ssn을 저장하는 형태이기 때문이다.
> 만약 반대로 매핑하였다면 EMPLOYEE의 FK로 DEPARTMENT의 PK인 Dnumber을 추가하였을 것이다. 이렇게 되면 대부분의 EMPLOYEE의 FK Dnumber는 NULL 값을 가지게 되어 공간을 낭비한다.

- MANAGES의 simple attributes를 DEPARTMENT에 추가: **Start_date** as **Mgr_start_date**

Binary 1:N

1. WORKS_FOR
    a. 참여하는 관계: EMPLOYEE & DEPARTMENT
    b. EMPLOYEE가 N쪽 엔티티
    c. 따라서 DEPARTMENT의 PK를 EMPLOYEE의 FK로 추가: **Dnumber** as **Dno**
2. CONTROLS
    a. 참여하는 관계: PROJECT & DEPARTMENT
    b. PROJECT가 N쪽 엔티티
    c. 따라서 DEPARTMENT의 PK를 PROJECT의 FK로 추가: **Dnumber** as **Dnum**
3. SUPERVISION
    a. 참여하는 관계: EMPLOYEE (Supervisor, Supervisee)의 재귀적 관계

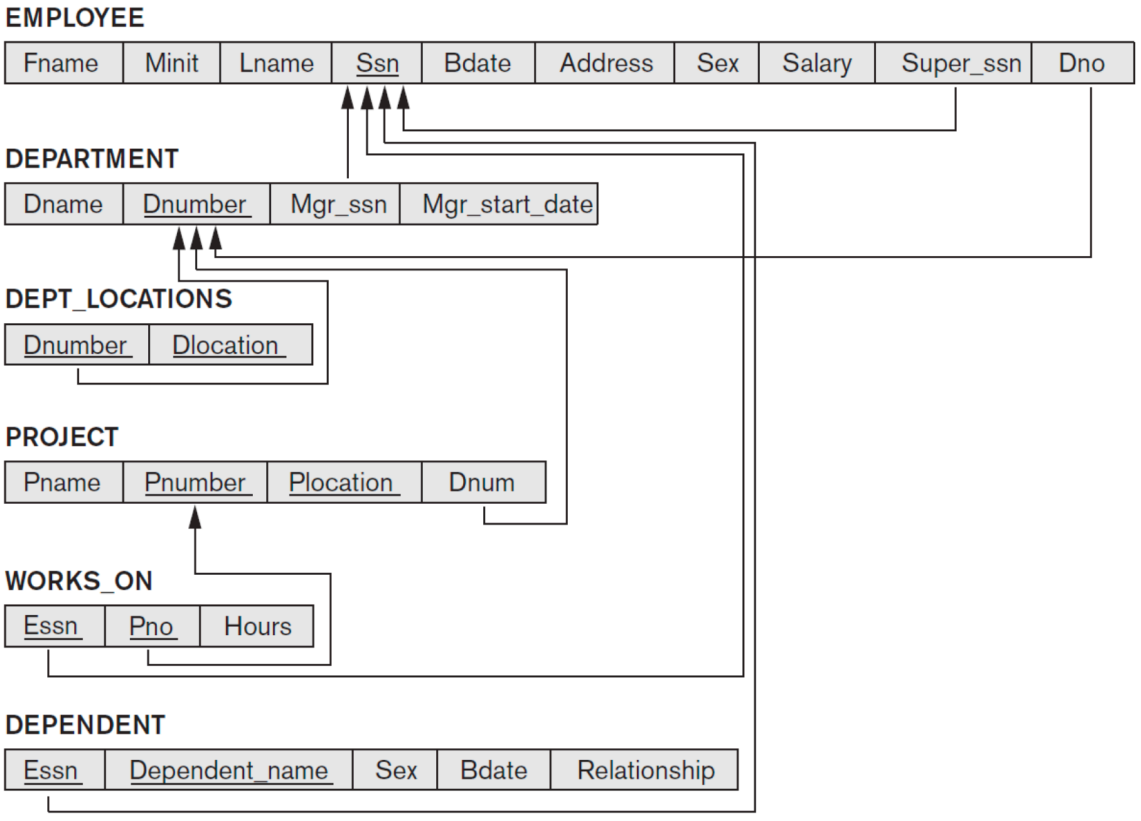
b. EMPLOYEE(Supervisee)가 N쪽 엔티티

c. EMPLOYEE(Supervisor)의 PK를 EMPLOYEE(Supervisee)의 FK로 추가: **Ssn** as **Super_ssn**

Binary M:N

WORKS_ON

- 새 관계 WORKS_ON 생성
- 참여하는 관계: EMPLOYEE & PROJECT
- EMPLOYEE와 PROJECT의 PK 추가: **Ssn** as **Essn**, **Pnumber** as **Pno**
- WORKS_ON의 simple attributes 추가: **Hours**
- **Essn** + **Pno**를 PK로 사용



▼ DB07

Relational Algebra 관계 대수
Basic set of operations for the relational model 관계형 모델을 위한 연산들의 집합

- Used for specifying retrieval requests 데이터를 요청하기 위해 사용
- 요청한 결과 데이터는 새로운 relation 관계로 표현된다.

SELECT

$\sigma_{<\text{selection condition}>}(R)$: 관계 R에서 선택 조건 c를 만족하는 튜플을 선택

- 선택 조건 c는 Boolean expression 부울식으로 표현됨
  - <attribute name> <comparison op> <constant value>
  - <attribute name> <comparison op> <attribute name>
- SELECT의 결과로 나타나는 관계는 c를 만족하는 튜플만 포함한다.

e.g. $\sigma_{\text{Dno}=4}(\text{EMPLOYEE})$

e.g. $\sigma_{\text{Salary}>30000}(\text{EMPLOYEE})$

$\sigma_{(\text{Dno}=4\ \textbf{AND}\ \text{Salary}>25000)\ \textbf{OR}\ (\text{Dno}=5\ \textbf{AND}\ \text{Salary}>30000)}(\text{EMPLOYEE})$

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

PROJECT

$\pi_{<\text{attribute list}>}(R)$: Attribute list L에 명시된 속성들만 테이블에 표시

PROJECT의 결과로 나타나는 관계는 distinct tuples 중복되지 않는 튜플의 집합이므로, 기존 테이블보다 적은 튜플이 표시될 수도 있다.

$\pi_{\text{Lname, Fname, Salary}}(\text{EMPLOYEE})$

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

Relational Algebra Expressions 관계 대수식

관계 대수 연산을 연속적으로 적용할 수 있다. 이를 통해 더 복잡한 쿼리를 표현할 수 있다.

e.g. “Retrieve the first name, last name, and salary of all employees who work in department number 5”: 자연어 쿼리 → $\pi_{\text{Fname,Lname,Salary}}(\sigma_{\text{Dno}=5}$(EMPLOYEE)) 관계 대수식

- DEP5_EMPS ← $\sigma_{\text{Dno}=5}$(EMPLOYEE)
- RESULT ← $\pi_{\text{Fname,Lname,Salary}}$(DEP5_EMPS)

중간에 결과를 담을 관계를 하나 더 만들어서 넘길 수도 있다.

결과 테이블의 속성명을 바꿀 수도 있다.

TEMP ← $\sigma_{\text{Dno}=5}$(EMPLOYEE)

R(First_name, Last_name, Salary) ← $\pi_{\text{Fname,Lname,Salary}}$(TEMP)

Set Operations 집합 연산

- UNION 합집합: $R_1 \cup R_2$

- INTERSECTION 교집합: $R_1 \cap R_2$
- SET DIFFERENCE (MINUS) 차집합: $R_1 - R_2$
- CARTESIAN PRODUCT (CROSS PRODUCT) 곱집합: $R_1 \times R_2$

예제

DEP5_EMPS ← $\sigma_{\text{Dno}=5}$(EMPLOYEE)

RESULT1 ← π_{Ssn}(DEP5_EMPS)

RESULT2(Ssn) ← $\pi_{\text{Super_ssn}}$(DEP5_EMPS)

RESULT ← RESULT1 ∪ RESULT2

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

Union Compatibility 합집합 호환성

집합의 두 관계는 같은 유형의 튜플들을 가져야 한다는 특성

집합 연산 ∪, ∩, -에 대해서:

- 두 관계 $R(A_1, A_2, ..., A_n)$, $S(B_1, B_2, ..., B_n)$은 같은 속성의 수 n을 가져야 한다.
- 두 관계의 대응되는 두 속성은 호환 가능한 범위 compatible domain을 가져야 한다: $\text{dom}(A_i) = \text{dom}(B_i)\ (1 \le i \le n)$

CARTESIAN PRODUCT Operation 곱집합 연산

$R \times S$ 또는 $R(A_1, A_2, ..., A_n) \times S(B_1, B_2, ..., B_m)$

하나의 관계의 모든 튜플과 나머지 관계의 모든 튜플의 조합 combination

$Q(A_1, A_2, ..., A_n, B_1, B_2, ..., B_m)$

- 속성의 수: $n + m$
- 튜플의 수: $n_R * n_S$

→ 곱집합 이후 특정 값을 만족하는 SELECT 연산을 수행하기 위해 사용된다.

예제

e.g. “Retrieve a list of names of each female employee’s dependents”

FEMALE_EMPS ← $\sigma_{\text{Sex}='\text{F}'}$(EMPLOYEE)

EMPNAMES ← $\pi_{\text{Fname,Lname,Ssn}}$(FEMALE_EMPS)

FEMALE_EMPS

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	1968-07-19	3321Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291Berry, Bellaire, TX	F	43000	888665555	4
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

EMPNAMES

Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

EMP_DEPENDENTS ← EMPNAMES × DEPENDENT

$$EMP_DEPENDENTS \leftarrow EMPNAMES \times DEPENDENT$$

EMP_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...

$ACTUAL_DEPENDENTS \leftarrow \sigma_{Ssn=Essn}(EMP_DEPENDENTS)$

$RESULT \leftarrow \pi_{Fname,Lname,Dependent_name}(ACTUAL_DEPENDENTS)$

ACTUAL_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...

RESULT

Fname	Lname	Dependent_name
Jennifer	Wallace	Abner

JOIN Operation

서로 다른 테이블의 튜플 간의 관계를 처리하는데 사용

e.g. “Retrieve the name of the manager of each department”

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

CARTESIAN PRODUCT 사용:

- ALL_PRODUCT ← DEPARTMENT × EMPLOYEE
- DEPT_MGR ← $\sigma_{\mathrm{Mgr_ssn}=\mathrm{Ssn}}$(ALL_PRODUCT)
- RESULT ← $\pi_{\mathrm{Dname},\mathrm{Lname},\mathrm{Fname}}$(DEPT_MGR)

JOIN 사용:

- DEPT_MGR ← DEPARTMENT$\bowtie_{\mathrm{Mgr_ssn}=\mathrm{Ssn}}$EMPLOYEE
- RESULT ← $\pi_{\mathrm{Dname},\mathrm{Lname},\mathrm{Fname}}$(DEPT_MGR)

> 장점: 쿼리 3줄 → 2줄 + 연산 속도 & 저장 공간 향상

THETA JOIN Operation

$R \bowtie_{<\text{join condition}>} S$: Join condition 조인 조건 c를 만족하는 모든 튜플의 조합

- CARTESIAN PRODUCT를 한 뒤 SELECT 한 것과 같음
- Join condition c:
  - <condition> AND <condition> AND... AND <condition>
  - <condition> = $A_i \theta B_j$
  - $\theta = \{=, <, \leq, >, \geq, \neq\}$

EQUIJOIN Operation

조인 조건이 = 인 경우. 조인의 결과 서로 다른 속성의 값이 같은 튜플이 포함된다.

NATURAL JOIN Operation

$R * S$: EQUIJOIN에서 중복된 두번째 값을 제거한다.

예제

e.g. “Combine each PROJECT tuple with the DEPARTMENT tuple that controls the project”

DEPT ← $\rho_{(\mathrm{Dname},\mathrm{Dnum},\mathrm{Mgr_ssn},\mathrm{Mgr_start_date})}$(DEPARTMENT)

PROJ_DEPT ← PROJECT * DEPT

조인 조건을 별도로 명시하지 않기 때문에 ρ로 rename하여 속성명이 같게 해준다.

> Two relations can be joined with more than one JOIN attribute 두 테이블은 하나 이상의 조인 속성을 통해 조인될 수 있다.

예제

1. "Associate each DEPARTMENT with its manager"

   DEPARTMENT $\bowtie_{\text{Mgr_ssn}=\text{Ssn}}$ EMPLOYEE

2. "Associate each EMPLOYEE with the department for which the EMPLOYEE works"

   EMPLOYEE $\bowtie_{\text{Dno}=\text{Dnumber}}$ DEPARTMENT

> 하나의 테이블이 조인의 양쪽에 쓰일 수 있다.

예제

"Retrieve the name of each EMPLOYEE and the name of its supervisor"

SUPERVISOR(Super_ssn, Sfname, Slname) ← $\pi_{\text{Ssn,Fname,Lname}}$(EMPLOYEE)

TEMP ← EMPLOYEE * SUPERVISOR

RESULT ← $\pi_{\text{Fname,Lname,Sfname,Slname}}$(TEMP)

Complete Set 완전 집합

Any relational algebra operation can be expressed as a sequence of operations from this set 모든 관계 대수식은 완전 집합의 연산의 연속으로 표현될 수 있다.

- {SELECT, PROJECT, UNION, SET DIFFERENCE, CARTESIAN PRODUCT}
- JOIN? : CARTESIAN PRODUCT 이후 SELECT로 대신 표현 가능
- INTERSECTION? : UNION 이후 SET DIFFERENCE로 대신 표현 가능

Relationally complete languages: 어떠한 새로운 쿼리 언어가 이 완전 집합에 포함된 관계 대수식을 표현할 수 있다면 이 언어는 relationally complete 관계적으로 완전하

다고 할 수 있다.

▼ DB08-1